3章
モノリスを分割する

2章では、マイクロサービスアーキテクチャへの移行をどう考えるかを探った。具体的には、それが本当に良いアイデアなのかどうか、また、良いアイデアであるなら、新しいアーキテクチャの展開が正しい方向に進んでいるかをどう確かめていくべきかを探っていった。

良いサービスとはどのようなものか、なぜ小さいサービスの方が良いのかについては既に説明した。しかし、そうした定義に従っていないアプリケーションが既に大量に存在しているかもしれないという事実を、私たちはどう扱えばよいのだろう？　ビッグバンリライトに着手することなく、そうしたモノリシックなアプリケーションを分解するにはどうすればよいだろうか？

本書の残りの部分では、マイクロサービスアーキテクチャを導入する際に役立つさまざまな移行パターンやヒントを探っていく。この章で見ていくのは、ブラックボックスなベンダーソフトウェアやレガシーシステム、あるいは継続して保守・進化させる予定のモノリスに対して機能するパターンだ。段階的にアーキテクチャを移行していくには、どうしたって、既存のモノリシックソフトウェアを継続して連携・利用できるようにしなくてはならない。

移行を段階的に行うという意志を忘れてはいけない。マイクロサービスアーキテクチャへの移行は小さなステップで進めよう。そして、そのプロセスから学び、必要に応じて考えを軌道修正しよう。

3.1　モノリスを変更すべきかどうか

最初に考慮しなければならないのは、移行において既存のモノリスの変更を計画するか（またはそれが可能か）ということだ。

既存のシステムを変更できるのであれば、さまざまなパターンを自由に使えるという面で、自由度は最も高くなる。しかし、状況によっては、厳しい制約のためにそれが難しいこともある。既存システムがソースコードを所有していないベンダー製品である可能性や、組織内からは失われてしまったスキル・技術で書かれている可能性もあるだろう。

また、心の弱さが既存システムの変更から目を背けさせる可能性もある。たとえば、現状のモノリスが悪い状態で、変更のコストがあまりに高い場合には、既存のシステムから手を引いて作り直したくなる可能性がある（先に述べたように、残念ながら容易にこの結論に達する組織が多い）。他には、既存のモノリスで多くの人々が作業をしている場合に、その邪魔をしてしまう懸念もある。「パターン：抽象化によるブランチ」など、この章で紹介するいくつかのパターンによって、こうした問題は軽減できる。しかし、それでもなお、他への影響が大きすぎると判断してしまうかもしれない。

ある忘れがたい出来事を話そう。何人かの同僚と一緒に、計算量の多いシステムのスケーリングを支援した時のことだ。基礎となる計算は、与えられたC言語のライブラリによって行われていた。私たちの仕事は、さまざまな入力を収集してライブラリに渡し、結果を取得して保存することだった。ライブラリ自体には問題が山積していた。メモリリークと恐ろしく非効率的なAPI設計は、主要な問題のうちの2つにすぎなかった。これらの問題を解決するために私たちはライブラリのソースコードを何カ月も要求し続けた。しかし、最後までそれは叶わなかった。

何年もたってから、私はプロジェクトのスポンサーを捕まえ、どうして基礎となるライブラリを変更させてくれなかったのかを尋ねた。その時、スポンサーはついにソースコードを失っていたことを認めた。しかし、恥ずかしくて私たちに伝えられなかったのだ。こんなことにはならないようにしよう。

できれば既存のモノリシックなコードベースで作業し、変更を行える立場にあることを願いたい。しかし、もしそれができないとすると、それは行き詰まりを意味してしまうのだろうか？　そんなことは全くない。そうしたときに役立

つパターンも存在する。そのうちのいくつかについては、この章で取り上げていく。

3.1.1　切り貼りか再実装か？

モノリス内にある既存のコードにアクセスできたとしても、新しいマイクロサービスに機能を移行する際に、既存のコードをどうするかは必ずしも明確ではない。コードをそのまま持って行くべきだろうか？　それとも機能を再実装すべきだろうか？

既存のモノリシックなコードベースが十分に整理されているなら、コード自体を移動することで大幅な時間を節約できるかもしれない。しかし、その場合には、モノリスからコードを**コピー**し、そして、少なくともこの段階では、モノリス自体からこの機能を削除しないことがとても重要だ。どうしてか？　機能を一定期間モノリスに残しておくことで、より多くの選択肢が得られるからだ。そうすることで、ロールバックできるポイントや両方の実装を同時実行する機会が得られる。そうして、移行がうまくいったことに満足したなら、モノリスからその機能を削除しよう。

3.1.2　モノリスをリファクタリングする

多くの場合、既存のコードベースはビジネスドメインの概念に基づいて整理されていない。そのことは、モノリスにある既存のコードを新しいマイクロサービスで利用する際の最大の障壁となる。既存のコードは、技術的な概念に基づく分類がより顕著なことが多い（あなたがこれまでに見たことのあるモデル、ビュー、コントローラーのパッケージ名を思い出してみよう）。この場合には、ビジネスドメインの機能を移動させるのが難しいかもしれない。既存のコードベースがその分類に一致していないことで、移動しようとしているコードを見つけることさえ困難な可能性があるからだ。

既存のモノリスをビジネスドメインの境界線に沿って再編成するというルートに進むのであれば、Michael Feathersの『レガシーコード改善ガイド』（翔泳社）[9]が間違いなくお勧めだ。同書では、**接合部（seam）**の考え方が紹介されている。接合部とは、既存の振る舞いを変更せずにプログラムの振る舞いを変えられる場所のことだ。変更したいコードの周りに接合部を定義し、接合部の新しい実装に取り組み、変更が行われた後にそれを入れ替えるというのが接合部の使い方だ。同書では、コードベースの整理を支援する方法として、接合部で安全に作業するためのテクニックが紹介されている。

接合部の考え方は多くのスコープに適用でき、1章で説明した境界づけられたコンテキストにもとてもよく適合する。したがって『レガシーコード改善ガイド』はドメイン駆動設計の考え方に直接言及している書籍ではないものの、この本にあるテクニックを使用することで、境界づけられたコンテキストの原則に沿ってコードを整理できる。

モジュラーモノリス？

既存のコードベースを理解し始めたら、検討する価値のある次のステップは、新たに特定した接合部を別のモジュールとして抽出し、モノリスを**モジュラーモノリス**にすることだ。モジュラーモノリスでは、デプロイユニットは依然として単一であるものの、それは静的にリンクされた複数のモジュールで構成される。これらのモジュールの正確な性質は、基盤となる技術スタックによって異なる。Javaの場合、モジュラーモノリスは複数のJARファイルで構成される。Rubyの場合は、それはgemライブラリのコレクションとなるかもしれない。

本書の冒頭で簡単に触れたように、モノリスをモジュールに分割して独立して開発できるようにすることで、マイクロサービスアーキテクチャの課題の多くを回避しつつ、多くのメリットを提供できる。これは、多くの組織にとってスイートスポットになる可能性がある。最終的にはマイクロサービスアーキテクチャに移行することを目的としてモノリスをモジュラーモノリスに分解し始めたものの、モジュラーモノリスで問題のほとんどを解決できることが分かったという話を、これまでに複数のチームから聞いたことがある。

段階的にリライトする

機能の再実装へ踏みきる前に、まず既存のコードベースを調査し改善するというのが、私の定石だ。前著『マイクロサービスアーキテクチャ』[25]でのアドバイスもこれらの方針に沿っていた。チームはその作業から十分な利益を得られ、そもそもマイクロサービスを必要としないと気付くこともある。

ところが、実際には、マイクロサービスに移行する前にモノリスをリファクタリングするというアプローチをとっているチームはほとんどない。代わりに、新しく作成するマイクロサービスの責務を特定すると、その機能を新たにクリーンルーム実装[†1]するというのが一般的なようだ。

[†1] 訳注：先行実装のソースを見ずに仕様を実装すること。https://wiki.c2.com/?CleanRoomImplementation

しかし、機能の再実装を始めると、ビッグバンリライトに関連した問題を繰り返す危険性があるかもしれない。重要なのは、一度に小さな機能だけをリライトし、リライトした機能を定期的に顧客に提供することだ。サービスの動作を再実装する作業が数日から数週間であれば問題はないだろう。しかし、その期間がもし数カ月単位になるようなら、私は自分のアプローチを再検討するだろう。

3.2　移行パターン

私はこれまで、マイクロサービスへの移行の一環で使われる多くのテクニックを目にしてきた。この章の残りの部分では、これらのパターンがどこで役に立つのかや、どのように実装できるのかを探っていく。全てのパターンと同様に、これらのパターンも一概に「良い」アイデアというわけではない。ここでは、それぞれのパターンについて、あなたの文脈で意味があるかを理解するのに役立つ十分な情報を提供するよう心がけた。

それぞれのパターンの長所と短所を必ず理解すること。これらは一概に「正しい」方法とは言えない。

まずは、移行およびモノリスとの統合を可能にするテクニックを見ることから始めていく。主にはアプリケーションコードが存在する場所を扱う。最初は、最も有用で一般的に使われているテクニックの一つである、ストラングラーアプリケーションパターンについて説明する。

3.3　パターン：ストラングラーアプリケーション

システムをリライトする際によく使われるテクニックに、**ストラングラーアプリケーション**[†2]がある。Martin Fowlerは、ある種のイチジクが木の上のほうの枝に種を蒔くことにヒントを得て、このパターンを提唱した。イチジクは

木をつたわりながらだんだんと下りていき、地面に落ちて根を張り、徐々に元の木を包み込んでいく。元の木は、最初は新しいイチジクを支える構造物となり、最終的には枯れて腐る。そして、新しいイチジクだけがその場所に残ることになる。

[†2] http://martinfowler.com/bliki/StranglerApplication.html（日本語訳：https://bliki-ja.github.io/StranglerApplication/）[10]

新しいシステムが既存のシステムをまず包んで、それによってサポートされるという点が、ソフトウェアの文脈と上記の類似点だ。このパターンは、古いシステムと新しいシステムを共存させ、新しいシステムが成長する時間を与え、古いシステムを完全に置き換えていくというものだ。このパターンの主な利点は、すぐに分かるように、私たちの目標である新システムへの段階的な移行をサポートしていることだ。さらに、段階的に提供された新システムを引き続き利用しながらも、移行を一時的に中断したり、完全に中止したりすることも可能にする。

すぐに説明するが、このパターンを実装する際には、新しいアプリケーションアーキテクチャに向かって段階的なステップを踏むだけでなく、各ステップを簡単に元に戻せるようにして、各ステップのリスクを減らすようにする。

3.3.1　使い方

ストラングラーパターンは、従来はあるモノリシックシステムから別のモノリシックシステムに移行するのに用いられてきた。しかし、ここではモノリスから一連のマイクロサービスへの移行にこのパターンを用いる。このパターンを用いてマイクロサービスに移行を行う際には、モノリスからコードを実際にコピーするか、対象の機能を再実装する。さらに、対象の機能が状態の永続化を必要とする場合には、その状態をどうやって新しいサービスに移行させるか、そして、どうやって元に戻せるかを考慮する必要がある。データに関する側面については、4章で探っていく。

ストラングラーパターンは、図3-1に示すように、3つのステップで実装する。まず、既存のシステムから移行したい部分を特定する。このステップでは、2章で紹介したトレードオフのアクティビティなどを用いて、システムのどの部分に最初に取り組むべきかを判断する。次に、この機能を新しいマイクロサー

ビスに実装する。新しい実装の準備ができたら、モノリスからの呼び出しを新しいマイクロサービスに切り替えられるようにする。

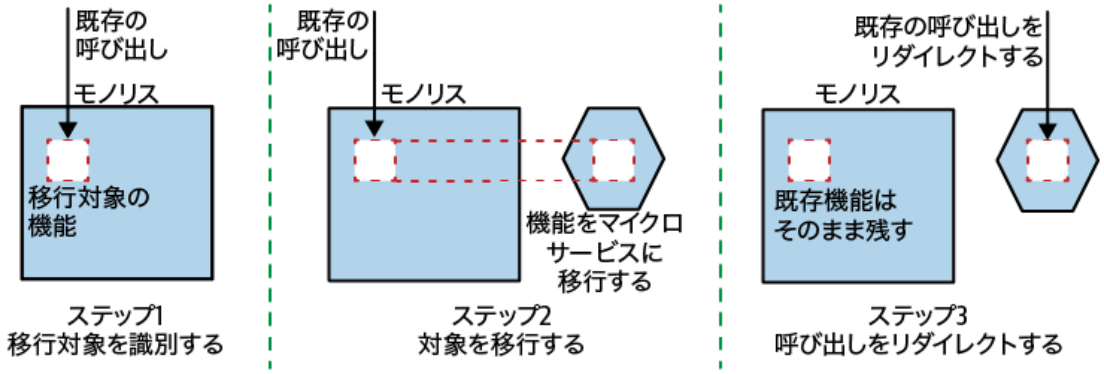

図3-1　ストラングラーパターンの概要

たとえ新しい機能が本番環境にデプロイされていたとしても、呼び出しが新しい機能にリダイレクトされるまでは技術的に有効でないという点は注目に値する。時間をかけてこの機能を実装していくことで、この機能を正しく作り込めるからだ。変更を本番環境に反映したとしても、それがまだ使用されないと分かっていれば安心だ。このことは、新しいサービスのデプロイと管理という2つの側面からうれしさを生む。新しいサービスにモノリスと同等の機能が実装できたなら、それが意図通りに動作しているかを確かめるのに「パターン：同時実行」といったパターンも使用できる。

デプロイと**リリース**の概念を分けることは重要だ。ソフトウェアが特定の環境にデプロイされたからといって、実際に顧客に使用されているとは限らない。この2つを別の概念として扱うことで、最終的な本番環境で使われるようになる前のソフトウェアを検証できるようになり、新しいソフトウェアを公開するリスクを軽減できる。ストラングラーや同時実行、カナリアリリースなどのパターンは、**デプロイ**と**リリース**が独立した活動であるということを利用したパターンの一例だ。

ストラングラーパターンの重要なポイントは、新しいシステムに機能を段階的に移行できるだけでなく、必要に応じて、この変更をとても簡単にロールバックできるということだ。誰だって間違いを犯すものだ。だからこそ、できるだけ安く間違えられる（つまり、細かなたくさんのステップに分ける）だけでなく、間違いを素早く修正できる技術が必要となる。

抽出された機能がモノリスの他の機能から使われている場合は、そちらの呼び出し方法も変更する必要がある。章の後半では、そのためのテクニックをいくつか紹介する。

3.3.2　使いどころ

ストラングラーパターンを使うと、既存のシステムに変更を加えることなく、新しいサービスに機能を移せる。これは、既存のモノリスで他の人が作業している場合にもメリットがある。また、モノリスがサードパーティ製のソフトウェアやSaaSサービスなど、実質的にブラックボックスシステムである場合にも非常に有効だ。

図3-2に示すように、ある機能全体をまるごと抽出できる場合もある。この場合は、データに関する懸念は別として、抽出が大幅に単純化される。データに関する懸念については後述する。

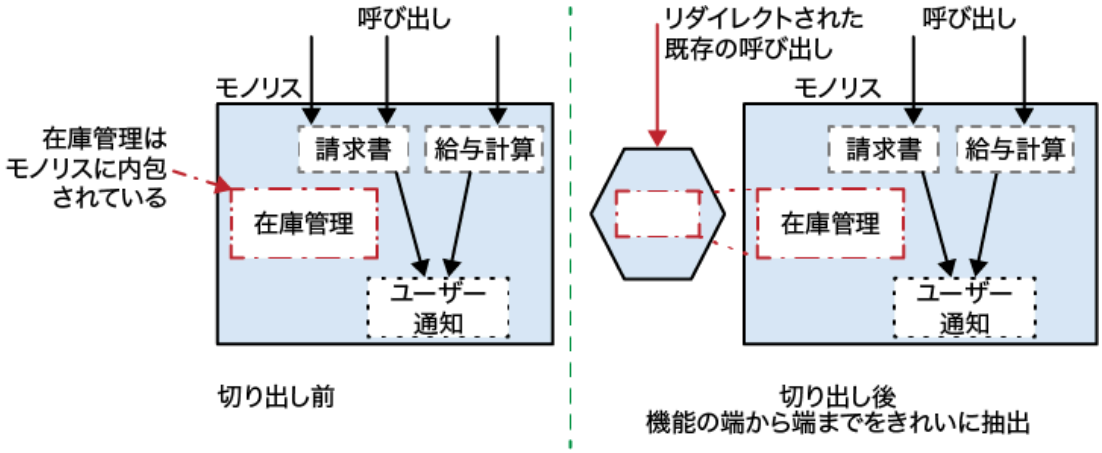

図3-2　在庫管理機能全体を単純に抽出

このように機能の端から端までをきれいに抽出するため、より大きな機能グループを抽出する方向に向かう傾向がある。その方が、プロセスが単純化するからだ。しかし、これは難しい綱渡りを生じさせる。より大きな機能のスライスを抽出することは、統合時のいくつかの課題を簡素化する一方で、より多くの作業を必要とするからだ。

抽出の単位を小さくしたい場合には、より「浅い」抽出を検討する必要があるだろう。図3-3では、「浅い」抽出の例として給与計算機能を抽出している。しかしながら、給与計算機能はモノリスに残っている他の機能（ここではユーザー通知機能）を利用している。

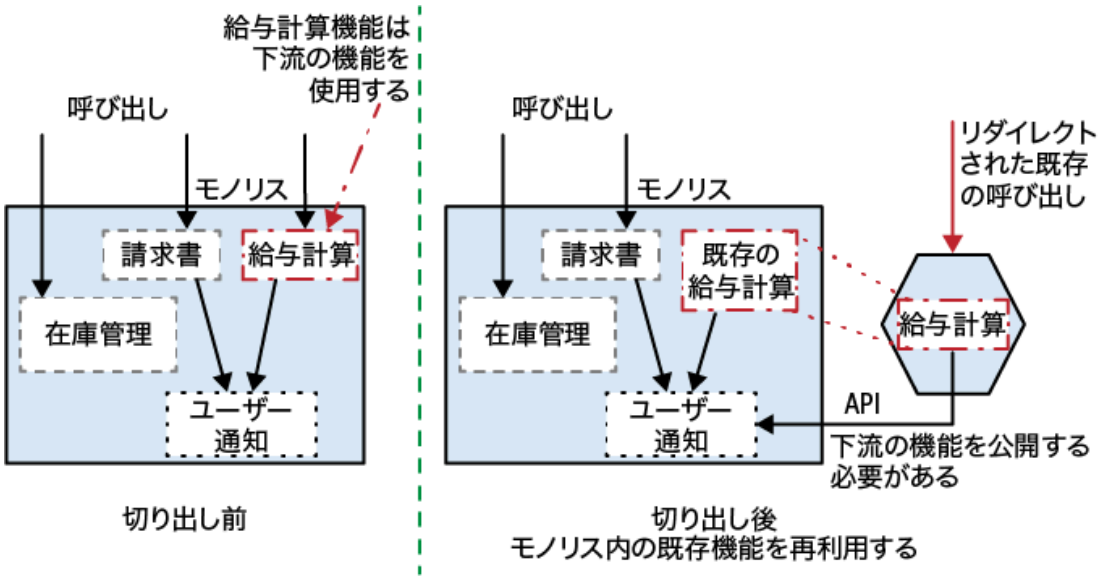

図3-3　モノリスを使用し続ける機能の抽出

この例では、新しいマイクロサービス向けにユーザー通知機能を再実装するのではなく、既存のユーザー通知機能をモノリスの外に公開することで、新しいマイクロサービスがユーザー通知を行えるようにしている。これを行うには、明らかにモノリス自体の変更が必要となるだろう。

また、ストラングラーが機能するには、移したい機能に対するモノリスへの呼び出しが移動先のサービスにはっきりと対応付けられている必要がある。たとえば、図3-4では、ユーザー通知機能を新しいサービスに移そうとしている。しかし、これは簡単にはいかない。ユーザー通知は既存のモノリスに対する呼び出しの結果として発生するものであり、システム外からの呼び出しをそのままリダイレクトはできないからだ。こうした場合には、「パターン：抽象化によるブランチ」のようなテクニックを使う必要があるだろう。

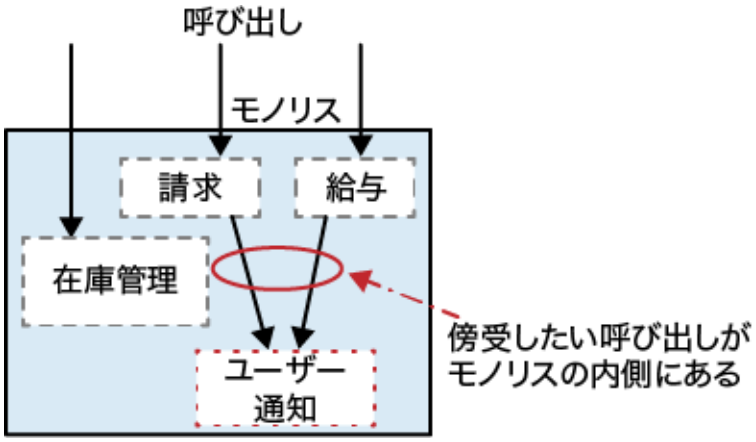

図3-4　移す機能が既存システムのより深い位置にある場合には、ストラングラーパターンはあまりうまく機能しない

また、既存システムの呼び出しが持つ性質も考慮する必要があるだろう。すぐに説明するが、HTTPのようなプロトコルはリダイレクトに非常に適している。HTTPには、プロトコル自体に透過的なリダイレクションの概念が組み込まれているからだ。そのおかげで、プロキシは受信リクエストの内容をはっきり理解でき、それを割り振れる。一方で、RPCのような一部のプロトコルはリダイレクトにあまり適さないかもしれない。プロキシ層で呼び出しの内容を理解して変換するために行わなくてはならない作業が増えるほど、この選択肢の実行可能性が低くなる。

こうした制限はあるものの、ストラングラーパターンがとても有用な移行テクニックであることは何度となく実証されてきた。気軽で簡単に段階的な変更にアプローチできるため、私がシステムの移行方法を検討する際には、ストラングラーパターンが最初の候補となることが多い。

3.3.3　例：HTTPリバースプロキシ

HTTPには興味深い機能がいくつもあるが、その中の1つに、リクエストの傍受やリダイレクトを呼び出し元のシステムが透過的になるような方法でとても簡単に行える機能がある。これは、HTTPインターフェイスを持つ既存のモノリスが、ストラングラーパターンを利用して移行するのに向いていることを意味している。

HTTPインターフェイスを公開している既存のモノリスの例を図3-5に示す。このアプリケーションはヘッドレスか、あるいは実際には上流のUIからこのHTTPインターフェイスが呼び出されているのかもしれない。どちらにして

も、ここでやりたいことは同じだ。それは、上流の呼び出しと下流のモノリスの間にHTTPリバースプロキシを挿入するということだ。

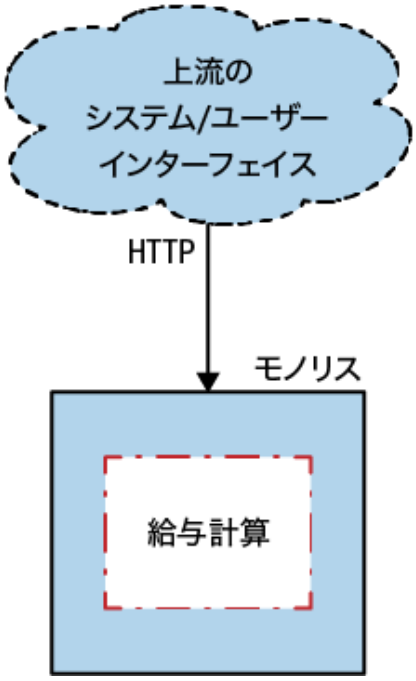

図3-5　ストラングラーパターンが実装される前のHTTP駆動モノリスの簡単な概要

ステップ1：プロキシを差し込む

再利用できる既存のHTTPプロキシがない場合には、図3-6に示すように**最初に**プロキシを用意することをお勧めする。この最初のステップでは、プロキシはどんな呼び出しもそのまま通過させるだけとする。

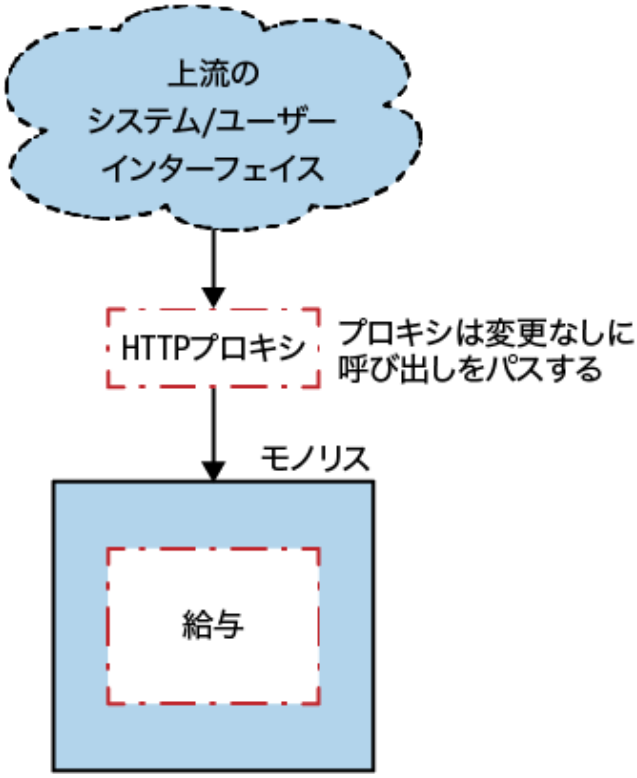

図3-6　ステップ1：モノリスと上流システムの間にプロキシを挿入する

このステップでは、まず上流からの呼び出しと下流のモノリスの間にネットワークホップを追加で挿入した場合の影響を評価し、新しいコンポーネントに対する必要な監視をセットアップし、そして基本的にはしばらくの間、様子を見る。レイテンシーの観点からは、全ての呼び出しの処理経路にネットワークホップと処理を追加することになる。まともなプロキシとネットワークがあれば、レイテンシーへの影響は最小限（数ミリ秒程度）に抑えられるだろう。しかし、レイテンシーへの影響が無視できないことが判明した場合には、いったん立ち止まって、ここで問題を調査する必要がある。

モノリスの前にプロキシが既に存在する場合には、このステップは省略できる。とはいえ、後でやれるからとたかをくくる前に、少なくともリダイレクトが意図通りに動くことくらいは確認しておくことをお勧めする。サービスを有効にする直前で、それができないと分かるような厄介なサプライズは起きてほしくないはずだ。

ステップ2：機能を移行する

プロキシを配置したら、次は新たなマイクロサービスの抽出を行う。この様子を図3-7に示す。

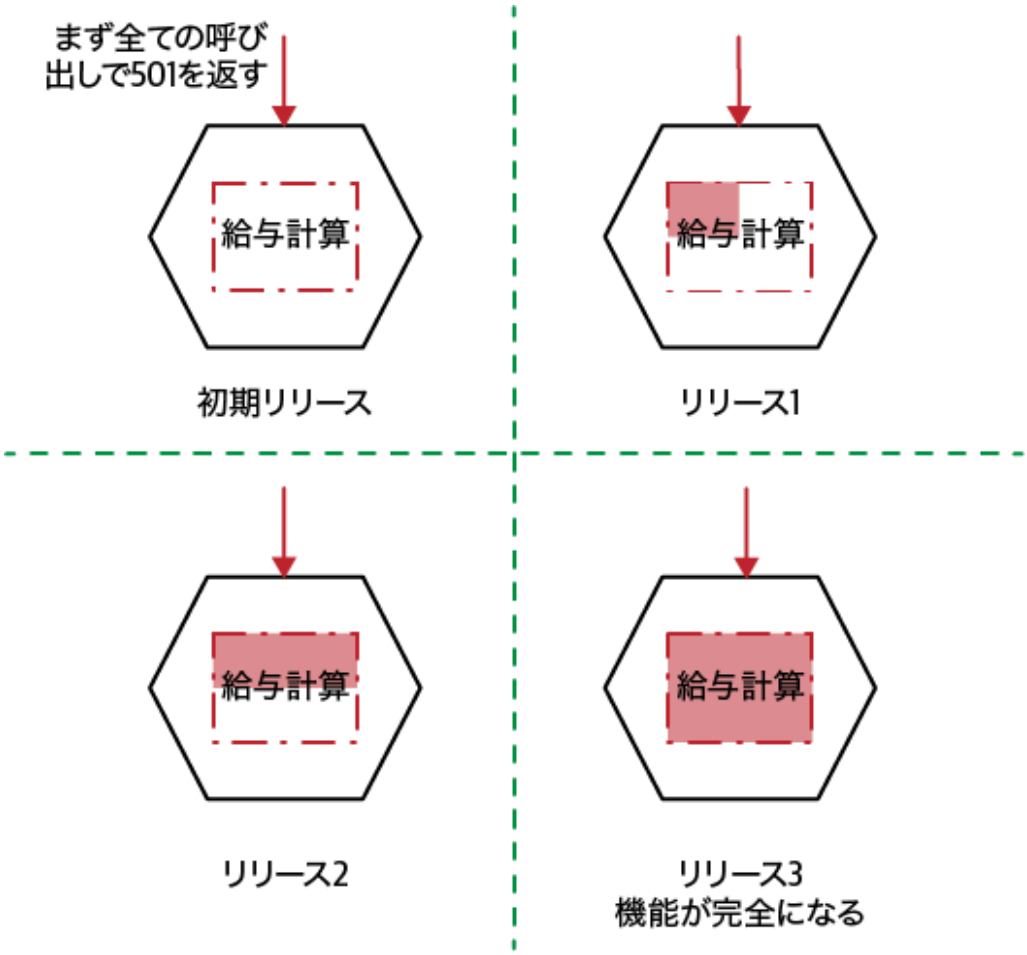

図3-7　ステップ2：機能の移行を段階的に実装する

このステップ自体は複数の段階に分けられる。最初に、機能を実装することなしに基本的なサービスを稼働する。サービスは対応する機能に対する呼び出しを受け入れる必要があるが、この段階では`501 Not Implemented`を返すだけとする。たとえこの段階であっても、私なら本番環境にこのサービスをデプロイする。そうすることで、本番環境へのデプロイプロセスに慣れ、サービスを実際にテストできるからだ。この時点では、上流の既存システムからの呼び出しをリダイレクトしていないため、新しいサービスはまだ**リリース済み**とは言えない。事実上、ここではソフトウェアのデプロイとリリースのステップを分離している。これは、後で改めて触れる一般的なリリース手法だ。

ステップ3：呼び出しをリダイレクトする

機能の移動が全て完了したら、図3-8に示すように、呼び出しをリダイレクトするようプロキシを再設定する。もし何らかの理由でこれが失敗した場合には、設定を元に戻す。ほとんどのプロキシでは、この設定はとても簡単に手早く行える作業であり、ロールバックもすぐに行える。

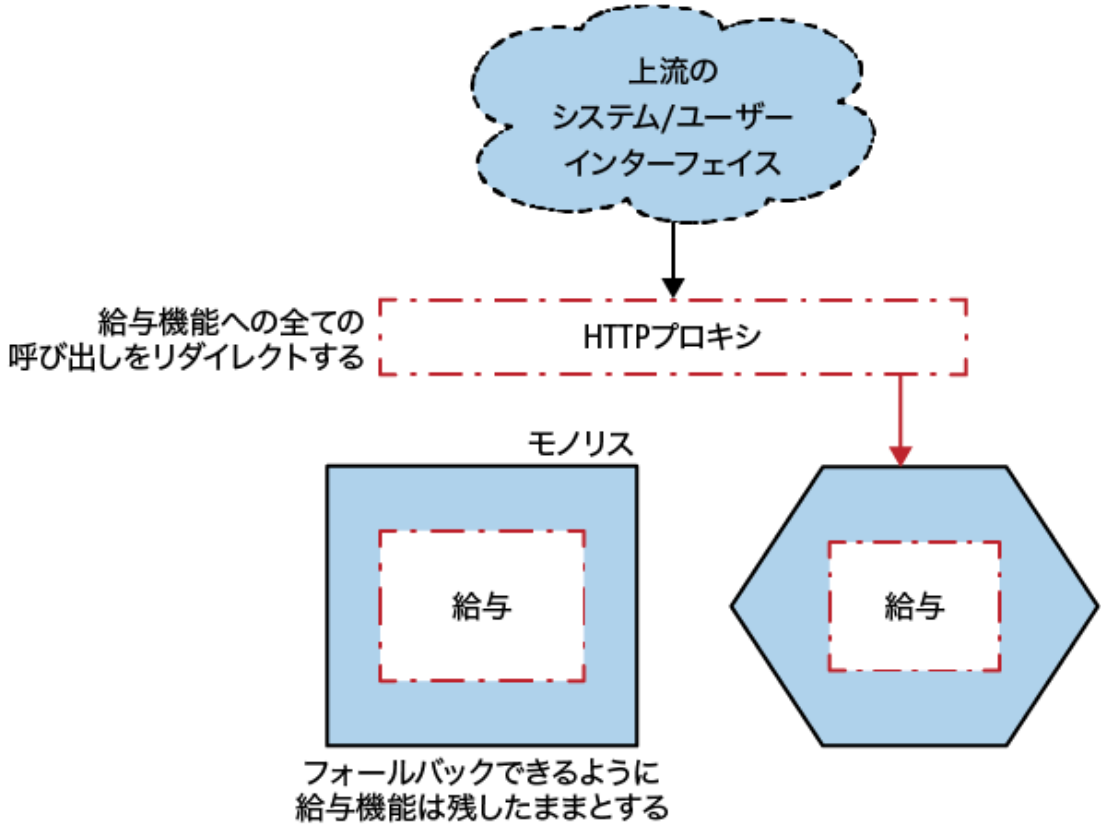

図3-8　ステップ3：給与機能への呼び出しをリダイレクトし、移行を完了させる

フィーチャーフラグ[†3]などを使ってリダイレクトを実装してもよいかもしれない。そうすることで、望んでいる構成状態をより明確にできるだろう。呼び出しをリダイレクトするためにプロキシを使うのは、この章の中で紹介するカナリアリリースや本格的な「パターン：同時実行」といったパターンを使って機能を段階的に公開していく際にもうってつけの方法だ。

[†3] 訳注：機能トグルとも呼ばれる。

3.3.4　データ？

今のところ、データについては触れてきていない。しかし、もし図3-8で、移行した給与計算サービスが、モノリスのデータベースが保持しているデータにアクセスする必要があるとしたら、どうなるだろうか？　こうした場合の選択肢については、4章で詳しく説明する。

3.3.5　プロキシの選択肢

プロキシをどう実装するかは、モノリスが使用しているプロトコルに依存する。既存のモノリスがHTTPを使っている場合には、幸先がよい。HTTPは広くサポートされているプロトコルなので、リダイレクトを管理する豊富な選択肢がある。おそらく私ならNGINXのような専用のプロキシを選ぶだろう。NGINXはまさにこうしたユースケースを念頭に置いて作られており、十分にテストされた多くのリダイレクトメカニズムをサポートしており、かなり良いパフォーマンスを発揮するだろう。

リダイレクトの中には、よりシンプルなものもある。RESTリソースを利用する場合のような、URIパス周りのリダイレクトを考えてみよう。図3-9では、請求書リソース全体を新しいサービスに移動させており、これはURIパスから簡単に解析できる。

しかし、既存のシステムがリクエストボディのどこか（おそらくはフォームパラメーター）に呼び出す機能に用いる情報を埋め込んでいる場合には、リダイレクトルールはPOSTでパラメーターを切り替えられるようにする必要がある（これは可能ではあるものの、より複雑になる）。このような状況に陥った場合は、プロキシの選択肢をチェックして、これに対応しているかを確認する価値は十分にある。

傍受やリダイレクトの内容がより複雑な場合や、それらがよくサポートされていないプロトコルをモノリスが使用している場合には、自分で何かをコーディングしたくなるかもしれない。しかし、そのアプローチを取るかは、かなり慎重に判断すべきだ。私は以前にいくつかのネットワークプロキシを自作したことがある（1つはJavaで、もう1つはPythonだった）。しかしながら、どちらの状況でもプロキシは信じられないほど非効率なものとなり、システムに大きな遅延を追加させてしまった。ただし、これは何よりも私のコーディング能力に原因があったかもしれない。ともかく、最近では、より独自の振る舞いが必要になった場合には、専用のプロキシにその独自の振る舞いを追加することが多い。たとえば、NGINXではLuaで書かれたコードを使って独自の振る舞いを追加できる。

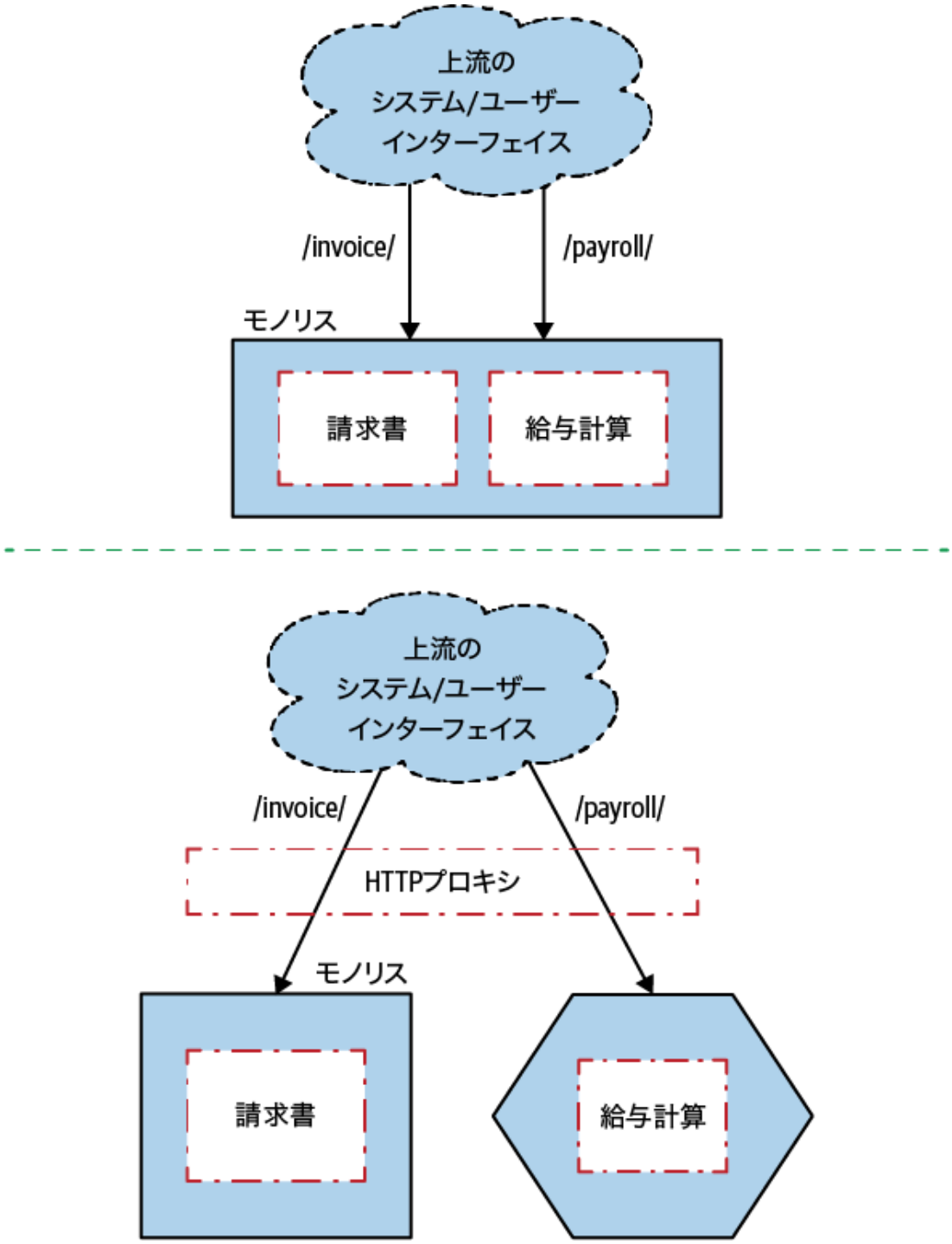

図3-9　リソースごとのリダイレクト

段階的に公開する

このテクニックを使うことで、アーキテクチャの変更を一連の小さなステップで行え、それぞれのステップをシステムの他の作業と並行して行える。これを図3-10に示す。

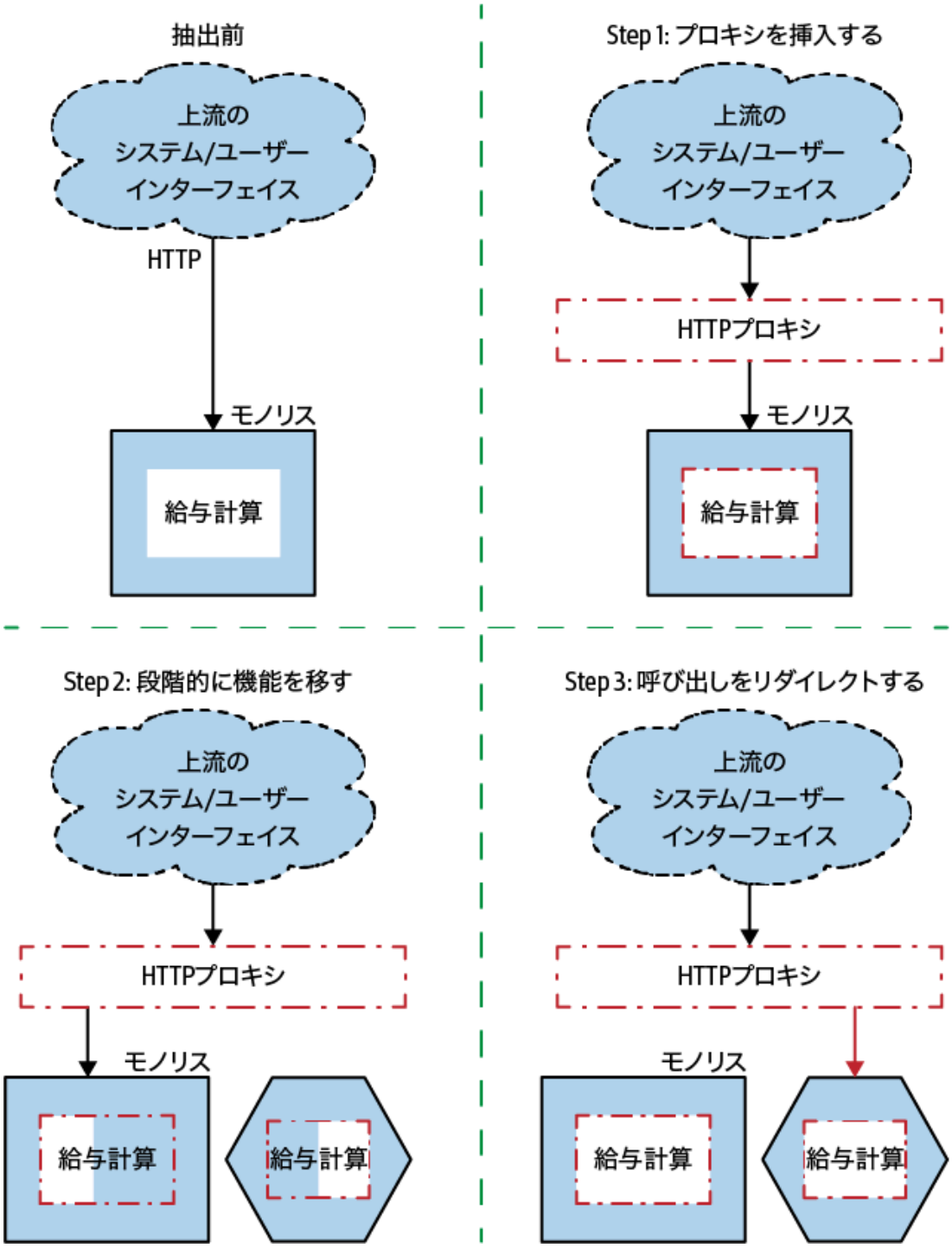

図3-10　HTTPベースのストラングラーパターンの実装例

新しい実装へ切り替える単位として、給与計算機能はまだ大きすぎると考えるかもしれない。その場合には、機能をより小さくスライスできる。たとえば、給与計算機能の一部だけを移行し、呼び出しを適切に迂回させることを検討できる。この場合、図3-11に示すように、動作の一部はモノリスに実装され、一部はマイクロサービスに実装される。ただし、モノリスにある機能とマイクロサービスにある機能の両方が同じデータセットを参照する必要がある場合には、このアプローチは問題を引き起こす可能性がある。共有データベースが必要となり、それがもたらす全ての問題が発生する可能性が高いからだ。

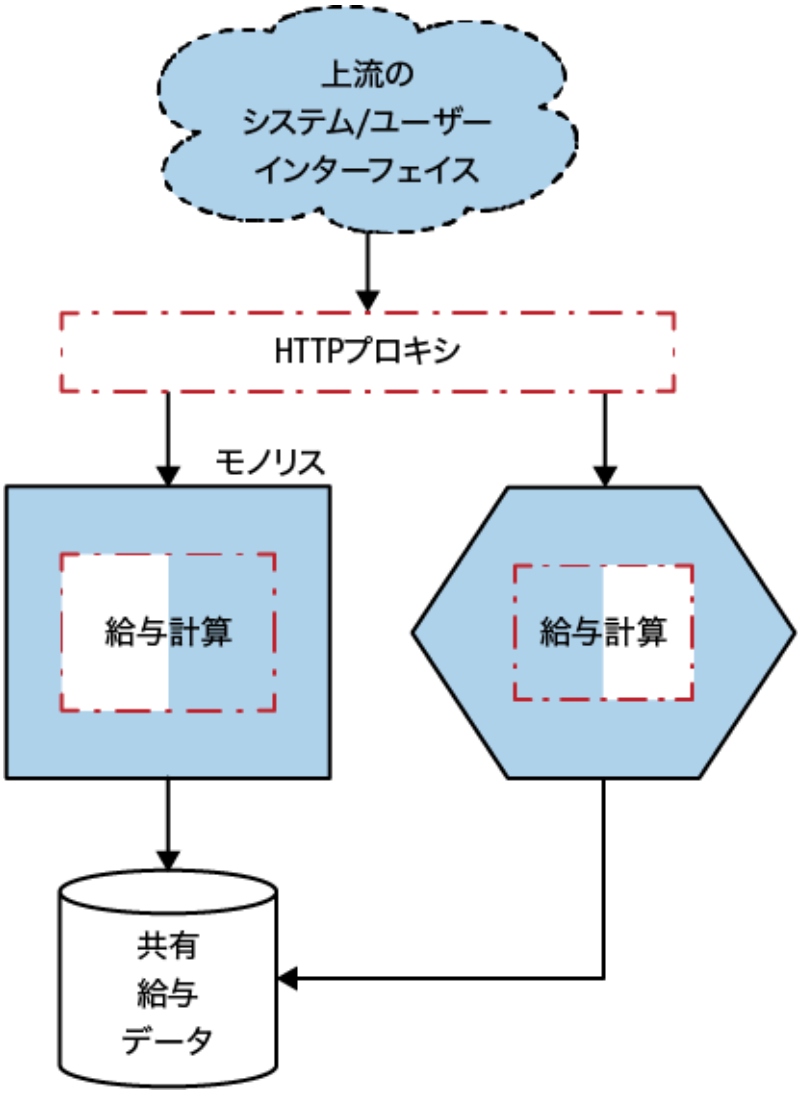

図3-11　給与計算機能の一部だけをスライスして移行する

作業を全て止めて行うような、大掛かりなリプラットフォーム[†4]は必要ない。ストラングラーパターンを使うことで、移行を他のデリバリー作業と並行して作業できるような複数の段階に分けるのがずっと簡単になる。バックログを「フィーチャー」ストーリーと「技術的な」ストーリーに分けるのではなく、それらの作業を一緒に束ねよう。そうすることで、新しいフィーチャーを提供しながら、アーキテクチャを段階的に変更することをうまくやっていけるはずだ。

[†4] 訳注：クラウドサービスなどを利用する形にアーキテクチャを変更し移行を行うこと。

3.3.6　プロトコルを変える

プロキシを使えばプロトコルを変換することもできる。たとえば、モノリスがSOAPベースのHTTPインターフェイスを公開している場合に、新しいマイクロサービスではそれに代えてgRPCインターフェイスをサポートさせたいとする。この場合には、図3-12に示すように、リクエストとレスポンスをそれに応じて変換するようにプロキシを設定する。

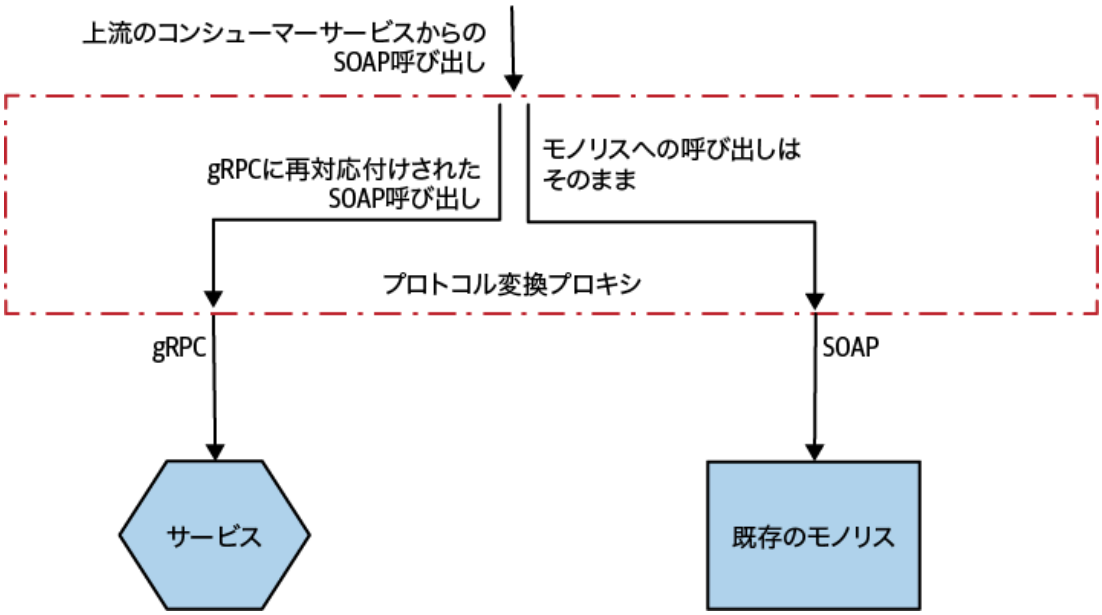

図3-12　ストラングラーパターンの移行の一部として、通信プロトコルを変更するためにプロキシを使う

しかしながら、このアプローチには懸念がある。それは、主にはプロキシ自体に複雑さとロジックが積み重なってしまうということだ。単一のサービスでは、これはそれほど悪いことではないように思える。しかし、複数のサービス向けにプロトコルを変換し始めると、プロキシで行われる作業がどんどん増えていくことになる。私たちは通常、サービスを独立してデプロイできるように最適化するが、複数のチームが変更しなくてはならない共有プロキシ層を持ってしまうと、そのせいで変更とデプロイのプロセスが遅くなってしまう可能性がある。新たな競合を起こさないように注意する必要があるからだ。マイクロサービスアーキテクチャを議論する際によく言われる「スマートエンドポイント、ダムパイプ」[†5]というマントラの通りだ。共有のミドルウェアレイヤーに機能が増えるとフィーチャー開発を遅らせる可能性がある。そのため、できるだけ共有のミドルウェアレイヤーに押しやる機能を少なくすべきだ。

[†5] 訳注：エンドポイントに高い機能を持たせ、通信はダムパイプ（メッセージのルーターとしてのみ動作する単純な機構）とすることで、メッセージ通信を軽量かつシンプルに保とうという考え方。

プロトコルを変更するのであれば、私なら、むしろサービス自体にそのマッピングを押し込むだろう。サービスが古い通信プロトコルと新しい通信プロトコルの両方をサポートするようにする。そして、図3-13に示すように、サービス内部で古いプロトコルへの呼び出しを新しいプロトコルにリマップする。これにより、他のサービスでも使われているプロキシ層の変更を管理する必要がなくなり、この機能を時間の経過とともにどう変化させるかをサービスが完全に制御できるようになる。こうすると、マイクロサービスをネットワークエンドポイント上の機能の集合体と捉えられるようになる。同じ機能を異なるコンシューマーに異なる方法で公開するといったことも可能だ。サービス内で異なるメッセージやリクエスト形式をサポートすることは、基本的には上流のコンシューマーの異なるニーズをサポートしているだけだ。

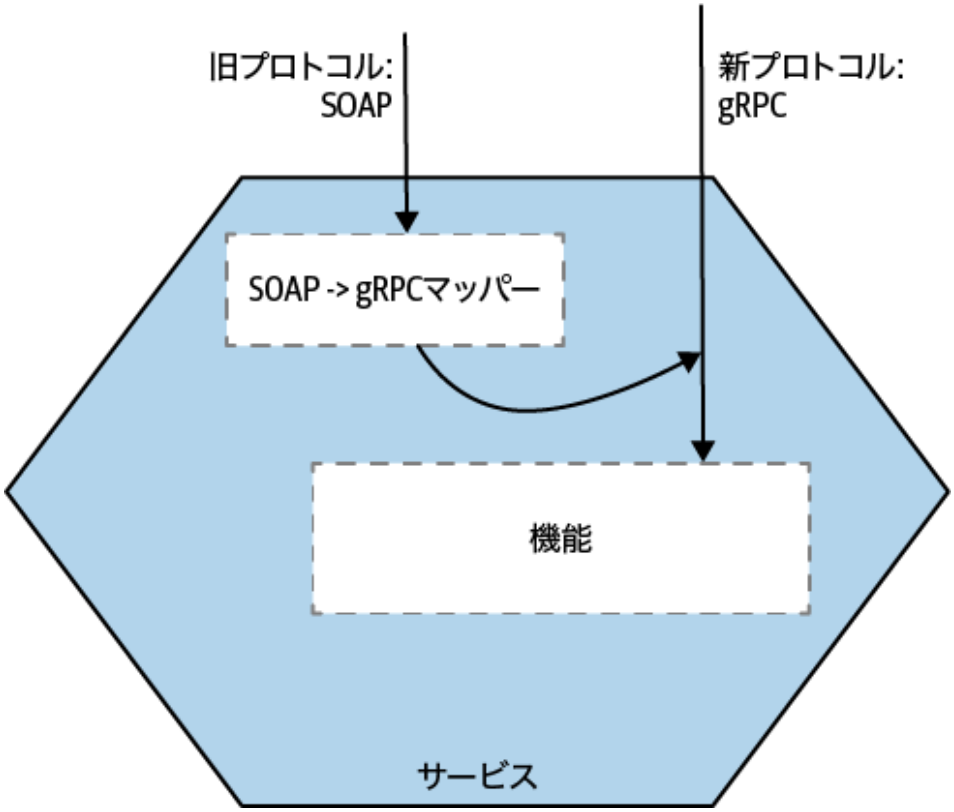

図3-13　プロトコルの種類を変更したければ、サービス側で機能を公開する際に複数のプロトコルをサポートできないか考えよう

サービス固有のリクエストとレスポンスのマッピングをサービスの中に押し込めることで、プロキシ層をシンプルに保ち、より汎用的にできる。さらに、サ

ービスが両方の種類のエンドポイントを提供することで、古いAPIを引退させなくてはならなくなる前に、上流のコンシューマーを移行させる時間も確保できる。

サービスメッシュ

この問題を解決するため、Squareはハイブリッドアプローチを採用した[†6]。非常に広範なエコシステムを備えかつ十分なサポートを持つオープンソースのRPCフレームワークであるgRPCによって、Squareは自社開発していたサービス間通信用のRPCメカニズムから移行することを決めた。この移行を可能な限り苦痛のないものにするには、各サービスで必要となる変更の量を減らす必要があった。そのため、Squareではサービスメッシュが活用された。

[†6] より詳しい説明は、Squareの開発者ブログに掲載されているSnow Pettersenによる記事 “The Road to an Envoy Service Mesh”（https://squ.re/2nts1Gc）[29]を参照してほしい。

サービスメッシュでは、それぞれのサービスインスタンスは専用のローカルプロキシを介して他のサービスインスタンスと通信する。これを図3-14に示す。プロキシの各インスタンスは、提携しているサービスのインスタンスごとに構成できるようになっている。また、コントロールプレーンを使うことで、これらのプロキシの集中制御と監視が行えるようにもなっている。中央にプロキシ層がないため、このアプローチでは、共有の「スマート」パイプを持つ落とし穴を回避できる。この事例で注目すべきは、アーキテクチャの進化の仕方である。Squareでは、LinkerdやIstioといった既存のソリューションに合わせるのではなく、オープンソースのプロキシ製品であるEnvoyを使用しつつも、ニーズに合わせて独自のサービスメッシュを作成することになった。

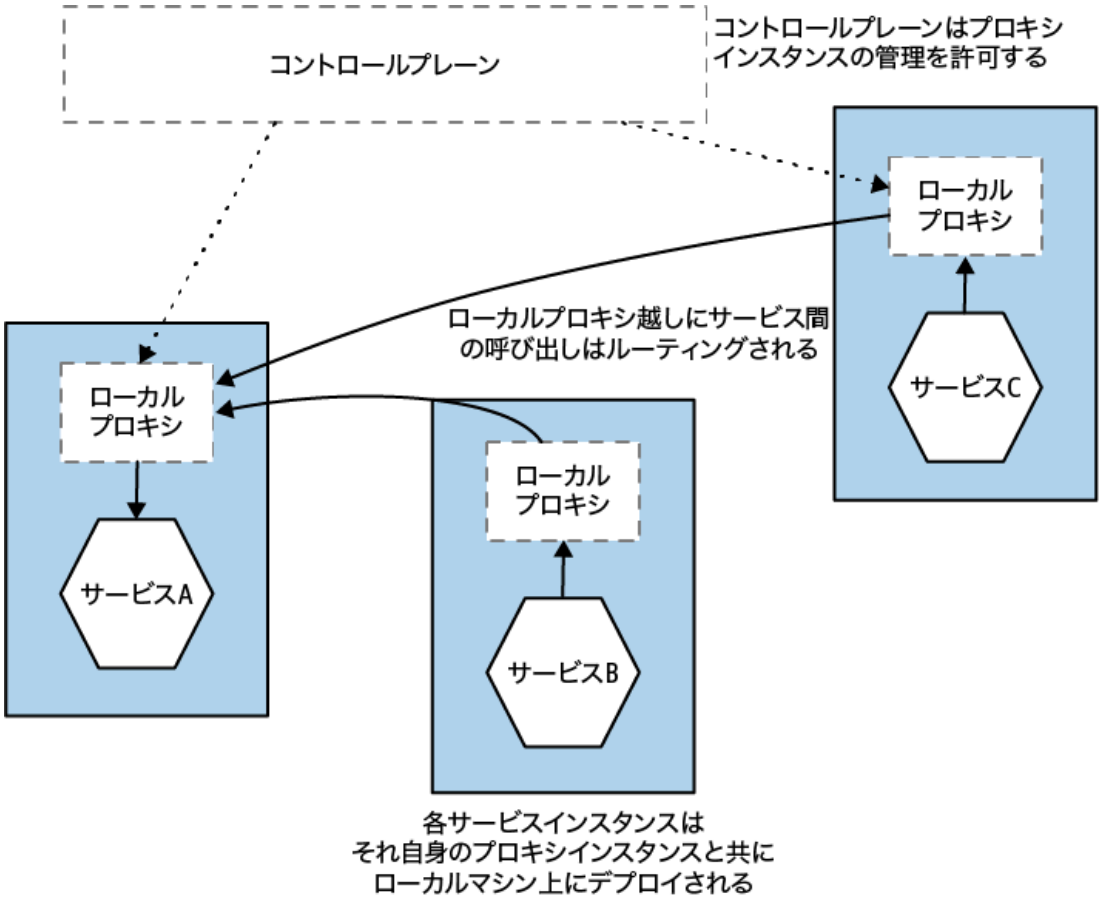

図3-14　サービスメッシュの概要

サービスメッシュの人気は高まっているし、コンセプト的にはそのアイデアはまさに的を射ているのだろう。サービスメッシュは、サービス間のコミュニケーションにおける一般的な懸念事項を処理する素晴らしい方法だ。しかし、サービスメッシュに関して私が心配なのは、非常に頭の良い人々が多くの仕事をしてきたにもかかわらず、この分野のツールが安定するまでに時間がかかっているということだ。Istioはこの分野における明確なリーダーのように見えるものの、この分野の唯一の選択肢とは言いがたい。そして、毎週のように新しいツールが登場している。私からの一般的なアドバイスは、サービスメッシュの分野が安定するまでは、できるならもう少し選択に時間を置いた方が良い、ということだ。

3.3.7　例：FTP

HTTPベースのシステムにストラングラーパターンを使用することについて長々と話してきた。そろそろ、他の通信プロトコルを傍受してリダイレクトする話をしてもよい頃合いだろう。スイスの不動産会社Homegateは、顧客が新しい不動産物件をアップロードする方法を変更するために、FTPでこのパターンを使用した。

Homegate社の顧客はFTP経由でリストをアップロードし、アップロードされたファイルは既存のモノリシックシステムによって処理されていた。同社はマイクロサービスへの移行に熱心で、FTPによるバッチ方式のアップロードではなく、当時推進されつつあったREST APIを使った新しいアップロードの仕組みもサポートし始めたいと考えていた。

ただし、Homegate社は顧客に影響を与えたくなかった。つまり、変更をシームレスに行いたかった。すなわち、当面の間は、FTPで顧客とシステムがやりとりする必要があったということだ。最終的に、同社は（FTPサーバーログの変更を検出することで）FTPのアップロードを傍受し、新たにアップロードされたファイルを新しいREST APIのリクエストに変換するアダプターに送ることにした。この様子を図3-15に示す。

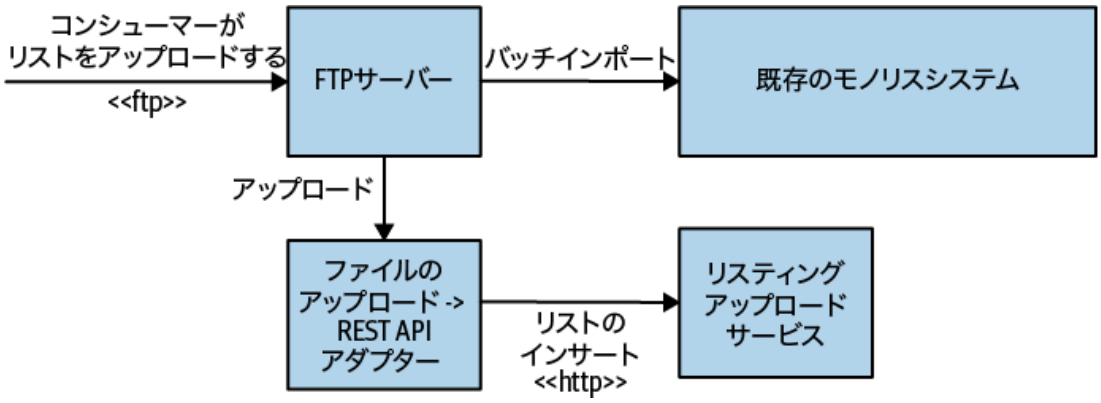

図3-15　FTPアップロードを傍受して、Homegateの新しいリスティングサービスに迂回させる

顧客のアップロードプロセス自体には変更はなかった。むしろ、アップロードを処理する新しいサービスがデータをより早く配信できたおかげで、顧客は物件情報を迅速に配信できるようになった。そして、後日、新しいREST APIも顧客に直接公開された。興味深いことに、この移行期間では、両方のアップロードメカニズムが有効になっていた。これによって、チームは2つのアップロードメカニズムが適切に動作していることを確認できた。これは、後で紹介する「パターン：同時実行」の好例だ。

3.3.8　例：メッセージ傍受

同期呼び出しを傍受することについては分かったが、では、モノリスが他の形式のプロトコルで呼び出されるような場合、たとえばメッセージブローカーを介してメッセージを受信するような場合はどうだろうか？　この場合でも、基

本的なパターンは同じだ。呼び出しを傍受し、新しいマイクロサービスにリダイレクトする方法が必要となる。主な違いは、プロトコル自体の性質だ。

コンテンツベースのルーティング

モノリスが多数のメッセージを受信していて、そのうちの一部を傍受する必要があるとする。図3-16にこれを示す。

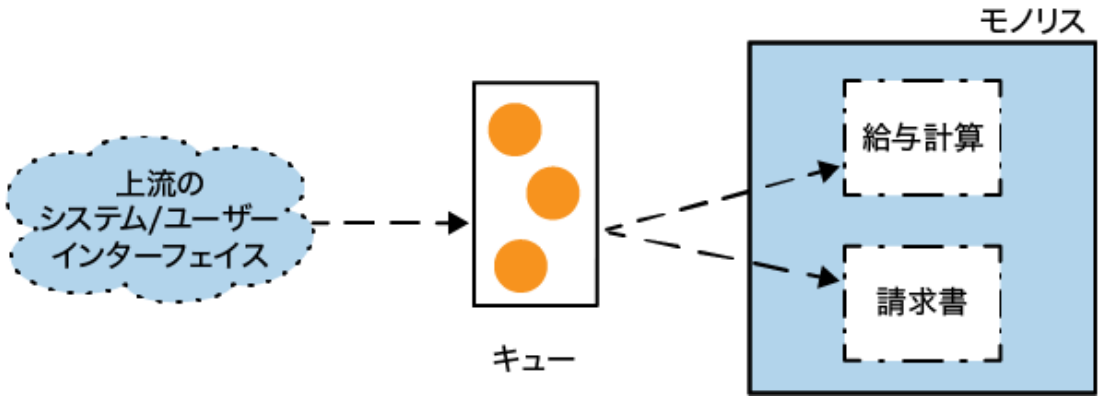

図3-16　キューを介して呼び出しを受けるモノリス

単純なアプローチとしては、図3-17に示すように、下流のモノリスに向けられた**全ての**メッセージを傍受して、適切にメッセージをフィルタリングするアプローチがある。これは基本的に “Enterprise Integration Patterns”（Addison-Wesley）[16]で説明されている**コンテンツベースのルーター**パターンの実装だ。

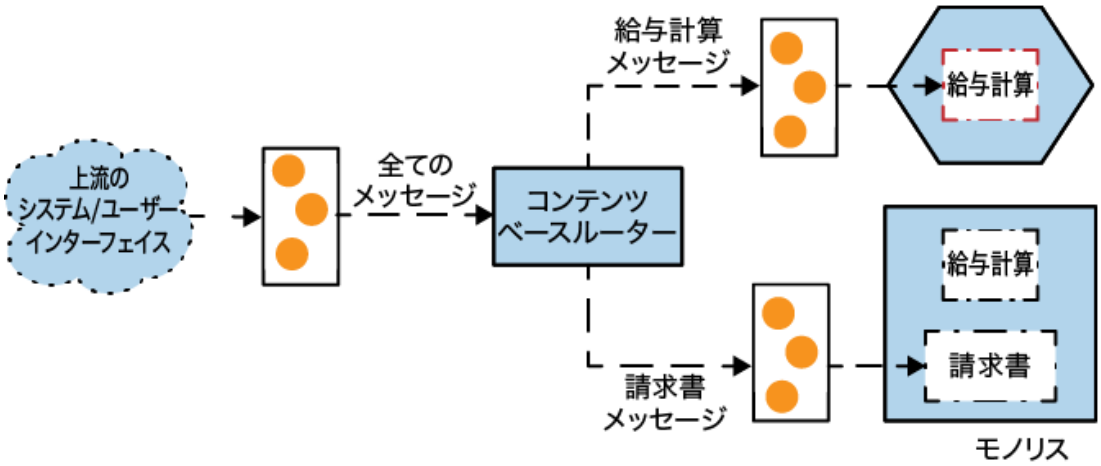

図3-17　コンテンツベースのルーターを使ってメッセージを傍受する

このテクニックでは、モノリスは変更せずに残しておけるものの、経路上に新たなキューを配置している。キューは独立して管理しなくてはならず、さらなるレイテンシーを生む可能性もある。メッセージングレイヤーに「スマートさ」

をどれだけ持たせるかということも課題だ。『マイクロサービスアーキテクチャ』[25]の4章では、サービス間のネットワークに「スマートさ」を持たせすぎることで引き起こされる課題について説明した。サービス間のネットワークをスマートにし過ぎると、システムが理解しにくくなり、変更が難しくなる可能性がある。その代わりに推奨したいのが、「スマートエンドポイント、ダムパイプ」というマントラだ。コンテンツベースのルーターが「スマートパイプ」を実装していることはほぼ間違いない。システム間で呼び出しがどうルーティングされるかという点について、複雑さが増しているからだ。場合によって、これはとても有用なテクニックではあるが、良いバランスを見つけられるかはあなた次第となる。

選択的消費

他には、新しいサービスが受信するメッセージを無視するように、モノリスを変更するというやり方がある。図3-18に示す例では、新しいサービスとモノリスはキューを共有している。そして、何らかのパターンマッチングを行って、それぞれが関心のあるメッセージを待ち受ける。この種のフィルタリングは、メッセージベースのシステムでは非常に一般的な要件だ。たとえば、JMS（Java Message Service）であればメッセージセレクターを使ってこれを実装できるし、他のプラットフォームでも同等の技術を使って実装が可能だ。

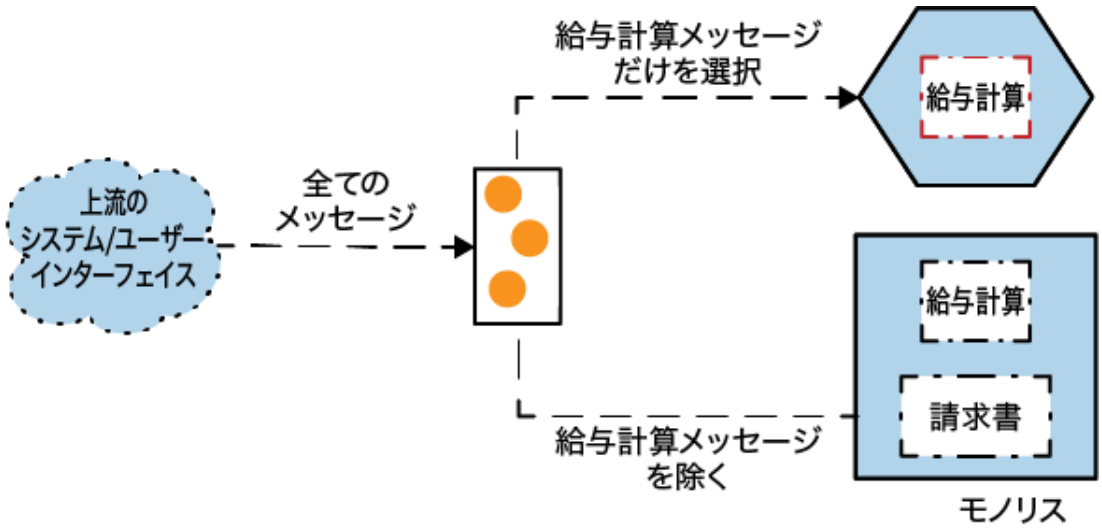

図3-18　マイクロサービスとモノリスでキューを共有し、それぞれが関心のあるメッセージだけを扱う

キューを新たに作成する必要はないものの、このフィルタリングアプローチにはいくつかの課題がある。まず、基盤となるメッセージング技術が、同一のキューを共同でサブスクライブできるようにはなっていないかもしれない（これ

は一般的な機能なので、もしサブスクライブできないとしたら私は驚くだろうが)。第二に、呼び出しをリダイレクトする場合、新しいサービスとモノリスの変更をかなりうまく調整しなければならない。モノリス側では新しいサービス向けの呼び出しを読み取るのを止め、サービス側ではそれらを拾う必要がある。同様に、呼び出しの傍受を元に戻すには、それら両方の変更をロールバックする必要がある。

同一のキューをサブスクライブするコンシューマーが増え、フィルタリングルールがより複雑になるほど、より多くの問題が発生するようになる。ルールの重複によって2つのコンシューマーが同じメッセージを受け取り始めてしまう状況や、逆に、一部のメッセージが完全に無視されてしまうといった状況は容易に想像できる。そのため、私が選択的消費を検討するのは、コンシューマーの数が少ないか、フィルタリングルールが単純な場合だけにしている。コンテンツベースのルーティングアプローチは、コンシューマーの種類が増えれば増えるほど意義を増すものであるが、前述のような潜在的な危険性、特に「スマートパイプ」問題に陥る可能性については注意が必要だ。

選択的消費やコンテンツベースのルーティングの厄介な点は、非同期のリクエスト・レスポンス型の通信を使用している場合に、クライアントが変更に気付かないように、リクエストをクライアントに返す必要があるということだ。メッセージ駆動型システムでの呼び出しのルーティングには他にも選択肢があり、その多くはストラングラーパターンによる移行を実装するのに役立つ。ここでは、“Enterprise Integration Patterns”（Addison-Wesley）[16]を素晴らしい情報源として強くお勧めする。

3.3.9　その他のプロトコル

この例から理解してもらえているだろうが、既存のモノリスへの呼び出しを傍受する方法は、たとえプロトコルの種類が異なっていたとしても、たくさんある。モノリスがバッチファイルのアップロードによって駆動されている場合はどうだろうか？　その場合には、バッチファイルを傍受して、ファイルの中から傍受したい呼び出しを抽出し、転送する前にファイルから削除しよう。確かに、仕組みによっては傍受を難しくするものがあるし、HTTPなどを使用した方がはるかに簡単だ。しかし、クリエイティブに考えることで、ストラングラーパターンは驚くほど多くの状況で使用できるはずだ。

3.3.10　ストラングラーパターンのその他の例

ストラングラーパターンは、既存のシステムを段階的にリプラットフォームしようとしているあらゆる状況で非常に有用だ。そして、その用途はマイクロサービスアーキテクチャを実装するチームだけに限定されていない。このパターンは、Martin Fowlerが2004年に書き上げる以前から使われていた。以前勤めていたThoughtWorksでは、モノリシックアプリケーションの再構築によく使われていた。Paul Hammantは彼のブログでこのパターンを採用したプロジェクトの網羅的なリストを作成している[†7]。その中には、トレーダー向けのブロッターや航空会社の予約アプリケーション、鉄道会社の予約システム、求人広告ポータルが含まれている。

[†7] https://paulhammant.com/2013/07/14/legacy-application-strangulation-case-studies/

3.4　機能を移行しながら振る舞いを変える

本書では、既存のシステムをマイクロサービスアーキテクチャに段階的に移行する際に使用できるパターンを特に選んで紹介している。その主な理由の1つには、そうしたパターンを使えば、移行作業とフィーチャーの提供を同時に進められることがある。しかし、移行を進めているシステムの動作を変更したり、充実させたりしたい場合に発生する問題がまだある。

たとえば、既存の給与計算機能をモノリスの外に移すためにストラングラーパターンを利用するとしよう。ストラングラーパターンを使うことで、移行は複数のステップで行えるし、理論的には各ステップをロールバックすることも可能だ。新しい給与計算サービスを顧客に公開した後で問題が見つかった場合には、給与計算機能の呼び出しを古いシステムに戻すことになる。モノリスとマイクロサービスの給与計算機能が機能的に同等であれば、これはうまく機能する。しかし、移行の中で給与計算機能の動作を変更していた場合はどうなるだろうか？

給与計算サービスに、モノリスの同等の機能にはバックポートされていないバグフィックスが適用されていた場合、ロールバックするとシステムにバグが再発することになる。これは、給与計算サービスに新しい機能を追加した場合に

は、もっと問題になる可能性がある。ロールバックを行うには、顧客からフィーチャーを奪う必要がでてきてしまうからだ。

この問題を解決するのは簡単ではない。移行中の機能を途中で変更することを許容する場合には、ロールバックが困難になることを受け入れなければならない。移行が完了するまで変更を許可しない方がずっと簡単だ。移行に時間がかかればかかるほど、システムのこの部分に「フィーチャーの凍結」を強制することは難しくなる。システムの一部を変更したいという要望がなくなるのはまれだからだ。そのため、移行の完了までに時間がかかればかかるほど、「今のうちにこのフィーチャーを入れてしまえばいい」というプレッシャーが強くかかるようになる。1回に移行する機能が小さければ小さいほど、移行が完了するまでに受ける機能変更へのプレッシャーは少なくなるはずだ。

機能の移行では、移行中の振る舞いを変更することを抑止しよう。新しいフィーチャーやバグ修正は、可能なかぎり移行が完了するまで遅らせること。そうしないと、変更をロールバックする能力が低下してしまうだろう。

3.5　パターン：UI合成

これまで見てきたテクニックは、主にサーバー側での段階的な移行を後押しするものだった。しかし、ユーザーインターフェイスも、既存のモノリスや新しいマイクロサービスアーキテクチャから部分的に提供されている機能をつなぎ合わせる有益な機会を提供してくれる。

何年も前、ガーディアン紙[†8]のオンライン版を既存のCMSからJavaベースの独自プラットフォームに移行するプロジェクトの支援に関わっていた時のことだ。このプロジェクトは、紙版のリニューアルと同時に、オンライン版のルック＆フィールを刷新して公開する作業も行うものだった。私たちは段階的なアプローチを取ることを望んだ。そのため、既存のCMSから新しいWebサイトへのカットオーバーは、旅行やニュース、文化といった特定のバーティカル[†9]を対象に、部分的かつ段階的に行われた。そして、バーティカルごとの作業においても、移行をできるだけ小さな塊に分割する機会を模索した。

[†8] https://www.guardian.co.uk/

[†9] 訳注：バーティカルとは「垂直の」という意味。ある分野やジャンルに特化して構成されたサイトやサイト内の領域を「バーティカルメディア」あるいは単に「バーティカル」と呼ぶ。

このプロジェクトの作業では、最終的に2つの合成テクニックが役に立った。その後、他の企業との会話から、これらのテクニックのバリエーションが、多くの組織におけるマイクロサービスアーキテクチャの導入方法の中で重要な部分を占めているということが分かってきた。

3.5.1　例：ページ合成

ガーディアン紙のプロジェクトは、この後すぐに説明するように単一のウィジェットを公開することから始まったのだが、新しいルック＆フィールを表示するため、ほとんどはページベースの移行が計画された。作業はバーティカルごとに進められた。最初に公開したのは旅行領域のバーティカルだった。この移行期間中にサイトを訪問したユーザーは、サイト内の新しくなった箇所に行ったときだけ、異なるルック＆フィールを体験した。古いページの全リンクが新しいページ（URLが変更されていた）にリダイレクトされていることを確認する必要があったのは、大変な苦労だった。

数年後、Javaで作られたモノリスを別のものに移行するという技術的な変更を行ったとき、ガーディアン紙は再び、バーティカルごとに移行するという同じテクニックを用いた。この時には、新しいルーティングルールを実装するために、Fastlyコンテンツデリバリーネットワーク（CDN）が使用された。CDNは社内プロキシを使用するのと同じように効果的に使われた[†10]。

[†10] ガーディアン紙のGraham Tackleyから、私が最初に実装を支援した「新しい」システムは、現在のアーキテクチャに完全に置き換えられるまで、ほぼ10年続いたことを聞けたのはよかった。どうしてかというと、このウェブサイトの読者として、この期間中に全てが変わったことに全く気付いていなかったからだ。

オーストラリアのREAグループは、不動産物件のオンラインリスティングを提供している企業だ。同社では、商業用物件のリスティングと住宅用物件のリスティングはそれぞれ別のチームが担当しており、各チームはリスティング用の全チャネルを所有している。このように、体験の端から端までをチームが全て所有している場合には、ページベースで合成を行うアプローチは理にかなっている。REAはチャネルごとに微妙に異なるブランディングも採用している。このことは、なおさらページベースの分解が理にかなっていることを意味している。そうすることで、顧客グループごとに全く異なる顧客体験を提供できるからだ。

3.5.2　例：ウィジェット合成

ガーディアン紙のプロジェクトで最初に新しいプラットフォームへ移行する対象となったのは、旅行領域のバーティカルだった。旅行領域のバーティカルが選ばれた理由には、分類に関して興味深い課題があったこともあるが、それだけでなく、サイト内で取り立てて注目度の高い部分でなかったことがあった。つまり、私たちは何かを公開し、その経験から学びを得たいとは考えていたが、うまくいかなかったとしてもサイトの主要な部分に影響を与えないようにしたいとも考えていたのだ。

私たちは、世界中の魅力的な旅行先に関する取材記事が満載の旅行領域全体を一気に新しく変えるのではなく、システムを試すためのもっと控えめなリリースを望んでいた。そこで、新しいシステムを使って定義された、旅行先のトップ10を表示するウィジェットを1つデプロイした。このウィジェットは、ガーディアン紙の旧旅行ページ内に挿入された。その様子を図3-19に示す。私たちはこれをApacheのEdge-Side Includes（ESI）と呼ばれる技術を用いて行った。ESIは、Webページのテンプレートを定義しておき、Webサーバーでそのコンテンツを接合できる技術だ。

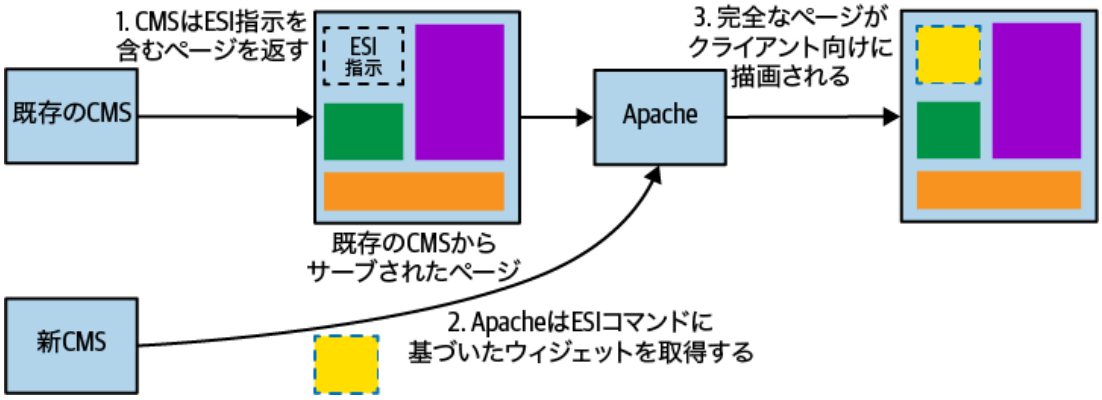

図3-19　Edge-Side Includesを使用して、新しいガーディアンCMSからコンテンツを接合する

最近では、純粋にサーバー側でウィジェットを接合することはあまり一般的ではないようだ。これは、ブラウザベースの技術がより洗練されて、ブラウザ自体（または後述するようなネイティブアプリ）でより多くの合成を行えるようになったことが大きい。ウィジェットベースのWeb UIでは、ブラウザ自体がさまざまなテクニックを駆使し、複数のウィジェットを読み込むために何度も呼び出しを行っていることが多い。これには、1つのウィジェットが（バックエンドサービスが利用できないなどして）ロードに失敗した場合でも他のウィジェットがレンダリングできることで、全体ではなく部分的なサービスの低下だけで済ませられるという利点がある。

ガーディアン紙のプロジェクトでは、最終的にほとんどの場合でページベースの合成を使用した。しかし、他の多くの企業ではバックエンドサービスをサポートするウィジェットベースの合成が多用されている。たとえば、現在はExpediaグループとなっているOrbitzでは、単一のウィジェットを提供することだけに特化したサービスが作成された[†11]。OrbitzのWebサイトは、マイクロサービスへの移行に先立って、既に個別のUI「モジュール」（Orbitz内での用語）に分割されていた。これらのモジュールは、検索フォームや予約フォーム、地図などを表示できるもので、図3-20に示すように、当初はコンテンツオーケストレーションサービスから直接提供されていた。

[†11] Steve HoffmanとRick FastによるGOTO 2016での講演 “Enabling Microservices at Orbitz”（http://bit.ly/2nGNgnI）参照。

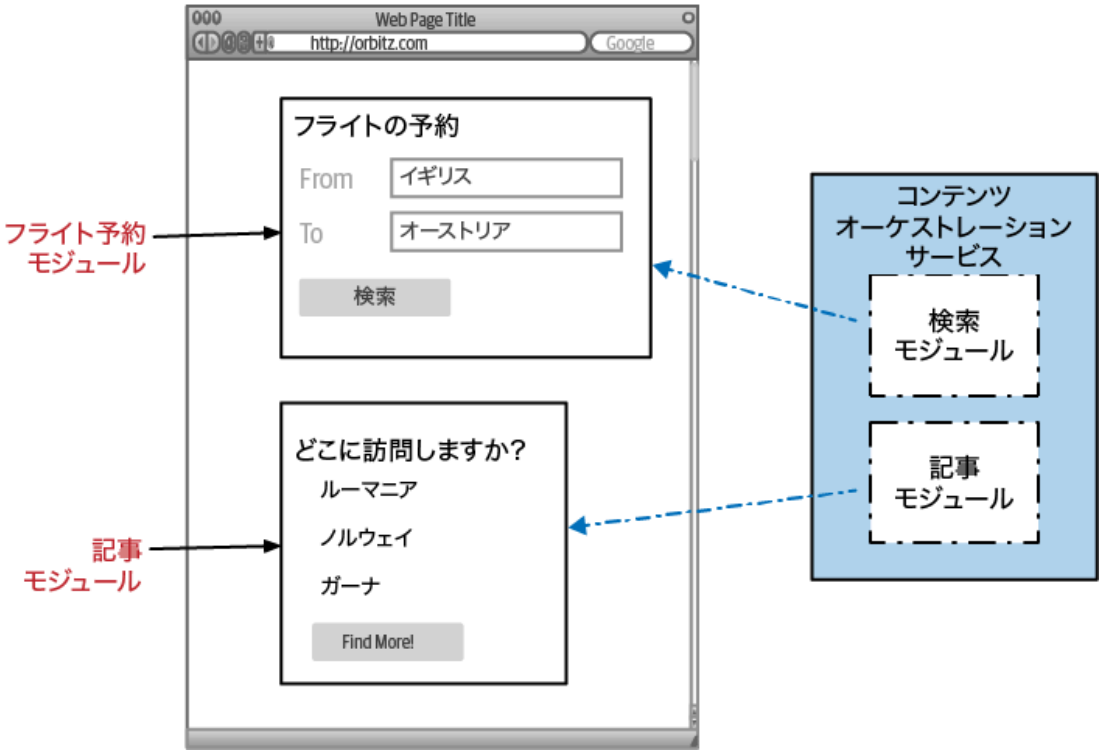

図3-20　マイクロサービスに移行する前、Orbitzのコンテンツオーケストレーションサービスは全てのモジュールを提供していた

このコンテンツオーケストレーションサービスは、実質的には巨大なモノリスだった。モジュールの開発チームは、モノリス内で行われる変更を調整しなければならず、それによって変更の公開に大きな遅れが生じていた。これは1章で説明したデリバリー競合問題の典型例だ。リリースサイクル高速化の取り組みの一環としてマイクロサービス化を試すことを決定したとき、Orbitzはモジュール境界に沿った分解に焦点を当て、それを記事モジュールから始めた。そして、コンテンツオーケストレーションサービスを、移行されたモジュールの責務を下流のサービスに委譲するように変更した。これを図3-21に示す。

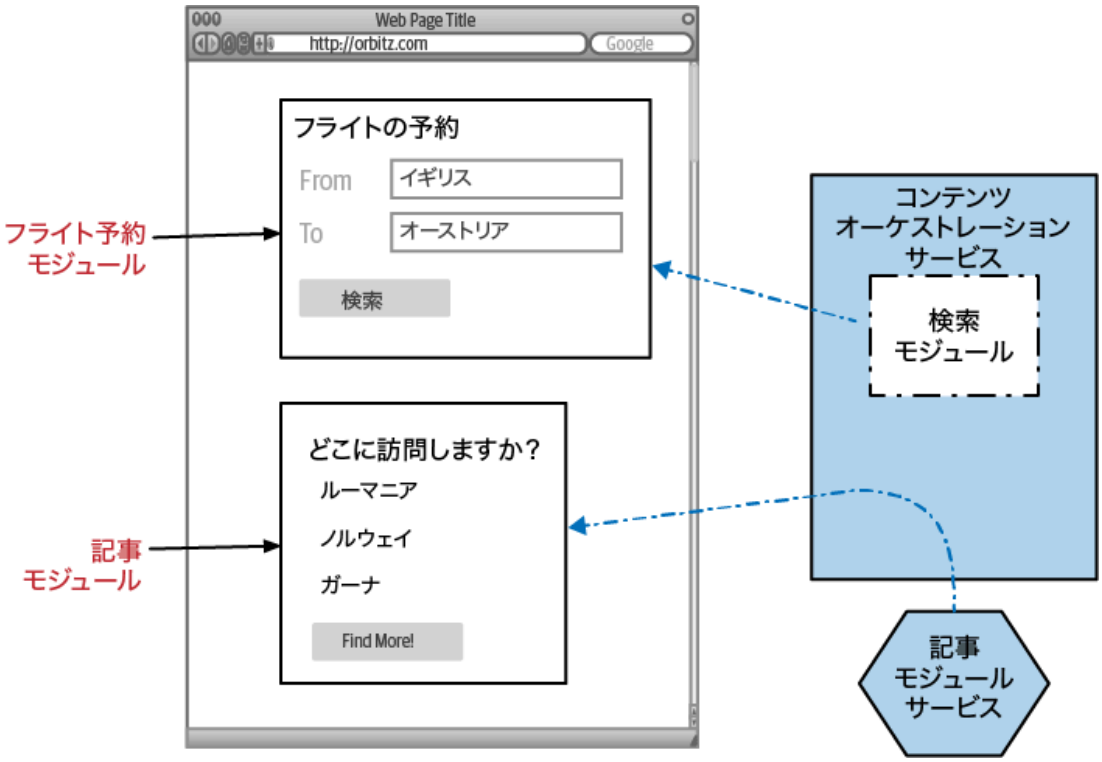

図3-21　モジュールは1つずつ移行され、コンテンツオーケストレーションサービスの責務を新しいサービスに委譲した

UIがこれらの境界に沿って既に分解されていたことは、この作業を段階的に実施しやすくした。さらに、分けられたモジュールに既に所有権の明確なラインがあったことで、他のチームに干渉することなく移行を実行することが容易になり、この移行はさらにうまく進んだ。

もちろん、全てのユーザーインターフェイスがすっきりとウィジェットに分けられるわけではない。とはいえ、それが可能な場合には、ウィジェットへの分解がマイクロサービスへの段階的な移行作業をずっと簡単にするということを覚えておこう。

モバイルアプリ

ここまでは主にWebベースのUIについて話してきたが、これらのテクニックの中にはモバイルベースのクライアントでも十分有効なものがある。たとえば、AndroidとiOSは、いずれもユーザーインターフェイスの一部をコンポーネント化する機能を提供しており、それによってUIの該当する部分を単独で作業したり、異なる方法で再結合したりすることが容易になっている。

ネイティブモバイルアプリケーションで変更をデプロイする際の課題の1つは、新しいバージョンを利用できるようにする前にAppleのApp StoreやGoogle Playストアにアプリケーションを提出し、審査を受ける必要があるこ

とだ。アプリケーションがアプリストアから承認されるまでの時間はここ数年で大幅に短縮されているものの、ソフトウェアの新しいリリースをデプロイするまでに時間がかかることに変わりはない。アプリ自体もまた、現時点ではモノリスとなっている。ネイティブモバイルアプリケーションの1つの部分を変更したい場合、アプリケーション全体をデプロイする必要があるからだ。また、ユーザーが新しいフィーチャーを体験するには新しいアプリをダウンロードしなければならないことも考慮しなければならない。これは、Webアプリケーションなら通常問題にならないことだ。Webアプリケーションであれば、一般に変更はブラウザにシームレスに配信されるからだ。

多くの組織では、新しいバージョンをデプロイすることなく、既存のネイティブモバイルアプリケーションに動的な変更を加えられるようにすることで、この問題に対処している。多くの組織が取り組んでいるのは、新しいバージョンのネイティブモバイルアプリケーションを常にデプロイしなくても、サーバー側に変更をデプロイすることによって、クライアントデバイスがすぐに新しい機能を取得できる方法だ。これは、組み込みのWebビューなどを使用するだけでも実現できるが、より洗練された技術を使用している企業もある。

SpotifyのUIは、iOSやAndroidアプリケーションを含め、全てのプラットフォームでコンポーネント指向を極めている。Spotifyでは、シンプルなテキストヘッダーからアルバムアートワーク、プレイリストに至るまで、目にするほとんど全てのものが独立したコンポーネントになっている[†12]。これらのモジュールのいくつかは、1つ以上のマイクロサービスによって支えられている。これらのUIコンポーネントの設定とレイアウトは、サーバー側で宣言的に定義するようになっているのだ。これにより、Spotifyのエンジニアは、アプリケーションの新しいバージョンをアプリストアに提出することなく、ユーザー向けのビューを変更し、その変更を迅速に公開できる。これにより、Spotifyでははるかに手早く新しいフィーチャーを試すことが可能となっている。

[†12] John Sundell “Building Component-Driven UIs at Spotify”（http://bit.ly/2nDpJUP）参照。

3.5.3　例：マイクロフロントエンド

帯域幅とWebブラウザの機能が向上するにつれて、ブラウザで実行されるコードも洗練されてきている。多くのWebベースのユーザーインターフェイスは、現在何かしらのシングルページアプリケーション（SPA）フレームワークを使用している。複数の異なるWebページから構成されるアプリケーションの考え方を取り払うことで、Webベースのアプリケーションは、全ての要素がシングルペインの中で実行される、より強力なユーザーインターフェイスを手に入れた。そして、以前はJavaのSwingのような「厚い」UI SDKを使うことでしか実現できなかった、より効果的なユーザー体験をブラウザ内で実現できるようになってきている。

インターフェイス全体をシングルページで提供するようになると、ページベースの合成はどうしたって考えられない。そのため、何らかのウィジェットベースの合成を行う必要がでてきた。Web用の一般的なウィジェットフォーマットを文書化しようとする試みがこれまでに多くなされてきたが、最近では、Web Components仕様の中で、ブラウザを超えてサポートされる標準的なコンポーネントモデルの定義が試みられている。しかし、（とりわけ）ブラウザのサポートがかなりの障害となっているため、この標準規格の普及には長い時間がかかっている。

VueやAngular、ReactといったSPAフレームワークの利用者は、Web Componentsが問題を解決してくれるのをただ待っているわけではない。代わりに、もともとブラウザペイン全体を所有する設計のSDKで構築されたUIを、どうやってモジュール化するかという問題について、多くの人々が取り組もうとしてきた。これが、**マイクロフロントエンド**と呼ばれるものへの後押しにつながっている。

マイクロフロントエンドとは、一見するとユーザーインターフェイスを独立して作業できるさまざまなコンポーネントに単に分解することのようだ。その点では、これは新しい考えではない。コンポーネント指向のソフトウェアは、私が生まれるよりも前から存在している。それよりも興味深いのは、人々がWebブラウザとSPAのSDK、そしてコンポーネント化を組み合わせて動作させる方法を研究していることだ。VueとReactの依存関係を衝突させずに、それらのパーツから単一のUIを作成し、その上で情報の共有を可能にするにはどうすればよいだろうか？

本書では、このトピックを深く掘り下げることは難しい。その理由の一つには、使用しているSPAフレームワークによってやり方が異なるということがある。しかし、シングルページアプリケーションを分解したいと思っても、あなたは孤独ではない。たくさんの人が、それを実現するためのテクニックやライブラリを共有してくれている。

3.5.4　使いどころ

システムのリプラットフォームを可能にする技術としてのUI合成は、機能の垂直スライス全体を移行できるため、非常に有効だ。しかし、UI合成を実現するには、新しい機能を安全に挿入できるように既存のユーザーインターフェイスを変更する能力が必要となる。本書の後半ではそうした合成技術について説明する。しかし、どの技術を使用できるかは、ユーザーインターフェイスを実装するために使用される技術の性質に多分に依存することに注意してほしい。古き良き時代のWebサイトであればUI合成は簡単だ。しかし、SPAでは複雑さが増し、実装アプローチに戸惑うことも多くあるだろう。

3.6　パターン：抽象化によるブランチ

有益な「パターン：ストラングラーアプリケーション」を機能させるには、モノリスの外側で呼び出しを傍受できなくてはならない。しかし、抽出したい機能が既存システムの奥深くにある場合は何が起こるだろうか？　前の例に戻って、図3-4の通知機能を抽出する場合を考えてみよう。

通知機能を抽出するには、既存システムに変更を加えなくてはならないだろう。これは大きな変更になる可能性がある。それと同時に、コードベースで作業している他の開発者を混乱させるだろう。ここには対立する関係がある。段階的に変更を加えていきたいと考えている一方で、コードベースの他の領域で作業している開発者をできるだけ混乱させたくないとも考えている。そうすると、自然と作業を早く完了させたいという気持ちに駆られるようになる。

既存のコードベースの一部を手直しするときには、ソースコードの別ブランチで作業を行うことがよくある。そうすることで、他の開発者の作業を中断することなく変更を行える。しかし、ブランチでの変更が完了したなら、その変更をマージしなければならない。これは、重大な問題を生むことがよくある。ブランチが長く存在すればするほど、そうした問題は大きくなる。ソースコードのブランチが長生きすることに関する問題については、継続的インテグレーションの原則に反していると言うこと以外、ここでは詳しく説明しない。“The

2017 State of DevOps Report（DevOpsの現状に関するレポート 2017年版）”[†13]で収集されたデータによると、トランクベースの開発（変更をメインラインで直接行い、ブランチを極力避ける方法）を採用し、短命のブランチを使用することがITチームのパフォーマンス向上に貢献していると示されていることも紹介しておく。ここでは、私は長命のブランチのファンではないということ、そして、そう考えているのは私一人ではないと言うにとどめておこう。

[†13] http://bit.ly/2pctNfn

私たちはコードベースに少しずつ変更を加えつつも、同じコードベースの他の箇所で作業している開発者の混乱を最小に抑えたいと考えている。**抽象化によるブランチ**[†14][†15]は、ソースコードのブランチに頼らずに、モノリスに少しずつ変更を加えることを可能にするパターンだ。ソースコードの別ブランチで作業を行う代わりに、抽象化によるブランチを利用して既存のコードベースに変更を加えることで、混乱を抑えつつも同じバージョンのコード内で実装を安全に共存させられる。

[†14] http://martinfowler.com/bliki/BranchByAbstraction.html（日本語訳：https://bliki-ja.github.io/BranchByAbstraction/）

[†15] 訳注：パターン名の由来は、抽象を用いて処理の分岐（ブランチ）を可能にすることから。

3.6.1　使い方

抽象化によるブランチは、5つのステップで構成される。

1. 置き換える機能の抽象を作る。
2. 作成した抽象を使用するように既存機能のクライアントを変更する。
3. 機能を改良した抽象の実装を新たに作る。本書の文脈では、この新しい実装が新しいマイクロサービスを呼び出すことになる。
4. 新しい実装を使用するように抽象を切り替える。

5. 抽象を後始末し、古い実装を削除する。

それでは、図3-4の通知機能をサービスに移行する手順を実際に見ていこう。

ステップ1：抽象を作成する

最初の作業は、置き換えるコードとそのコードの呼び出し元の相互作用を表す抽象を作ることだ。これを図3-22に示す。既存の通知機能が十分整理されて作られていれば、この作業はIDEのリファクタリング機能「インターフェイスの抽出」を適用するのと同じくらい簡単に行える。そうでない場合には、先に述べた接合部をまず抽出する必要があるだろう。このステップでは、電子メールやSMSといった通知メカニズムのAPIが呼び出されている箇所を調べなくてはならないかもしれない。このステップでは、そうした箇所を見つけ出し、他のコードが使用する抽象を作成することが必須となる。

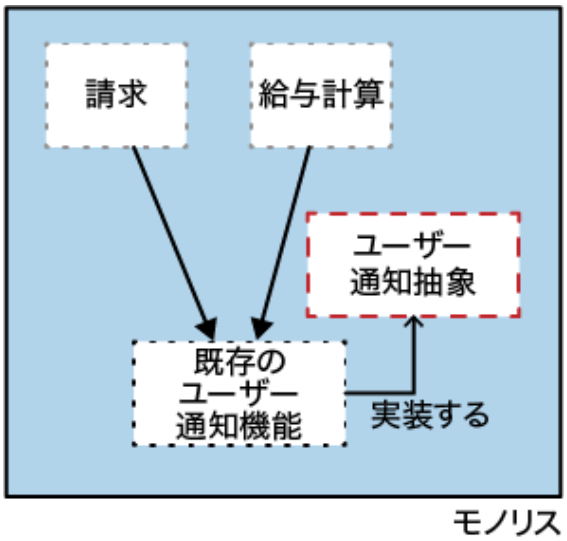

図3-22　ステップ1：抽象を作る

ステップ2：抽象を使用する

抽象を作成したら、次は新しい抽象ポイントを使用するように通知機能の既存クライアントをリファクタリングする必要がある。これを図3-23に示す。IDEのリファクタリング機能「インターフェイスの抽出」を使って自動でこれを行うことも可能だ。しかし、私の経験では、このステップは、対象の機能を呼び出している箇所を手作業で確認することなど、煩雑なプロセスを必要とすることが多い。このステップの良いところは、変更が小さく段階的ということだ。つまり、既存のコードに大きな影響を与えず簡単に行うことができるとも言える。この時点では、システムの動作に機能的な影響はないはずだ。

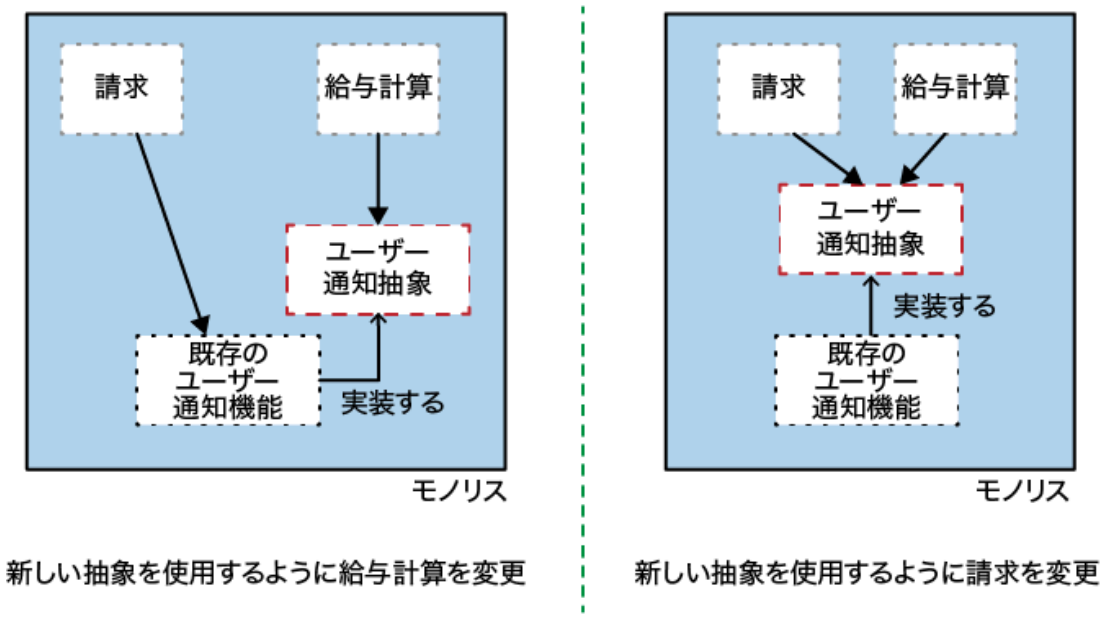

図3-23　ステップ2：既存のクライアントを、新しい抽象を使用するように変更する

ステップ3：新たな実装を作る

抽象を導入できたら、次は新しいサービスを呼び出す実装を開発し始められる。図3-24に示すように、モノリス内部における通知機能の実装は、ほとんどの場合は外部サービスを呼び出すクライアントになる。そして、通知機能の大部分はサービス側で実現される。

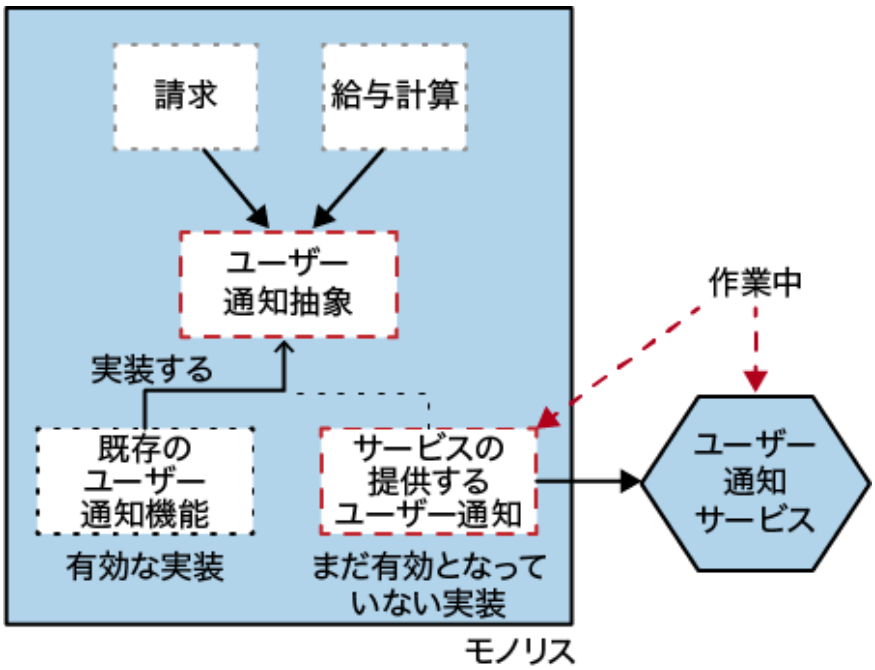

図3-24　ステップ3：抽象の新しい実装を作成する

このステップにおける重要な点は、コードベースには抽象に対する2つの実装が存在しているものの、システムで有効になっている実装はどちらか1つだけということだ。新しいサービスを呼び出す側の実装は、実際に通知を行えるようになるまでは実質的に休止状態となる。新しいサービスに全ての機能を実装するように取り組んでいる間は、抽象の新しい実装の方は`Not Implemented`のような未対応を表すエラーを返す可能性がある。もちろん、これは実装した機能のためにテストを書くことを止めるものではない。新しいサービスの検証が可能になることは、早期に統合することで得られる利点の1つだ。

「パターン：ストラングラーアプリケーション」と同じように、このプロセスの間でも、仕掛かり中のユーザー通知サービスを本番環境にデプロイできる。完成しているかは問題ではない。新しい通知機能の抽象の実装はまだ有効にはなっていないため、サービスは実際には呼び出されないからだ。しかし、それをデプロイすることで、私たちは本番環境でそれをテストし、実装した機能の一部が正しく動作しているかどうかを検証できる。

このステップはかなりの時間がかかる可能性がある。Jez Humbleは、継続的デリバリー用のアプリケーションであるGoCD（当時はCruiseと呼ばれていた）で抽象化によるブランチパターンを使用した事例を記事（http://bit.ly/2p95lv7）[18]に詳しく書いてくれている。GoCDでは、データベース永続化レイヤーを移行するために抽象化によるブランチパターンを使用した。iBatisの使用からHibernateへの切り替えは数カ月間続いたが、その間もアプリケーションは週2回のペースでクライアントに出荷されていたとのことだ。

ステップ4：実装を切り替える

新しい実装が正しく動作していることを確認できたら、抽象ポイントを切り替えて新しい実装を有効にし、古い機能が実行されないようにする。これを図3-25に示す。

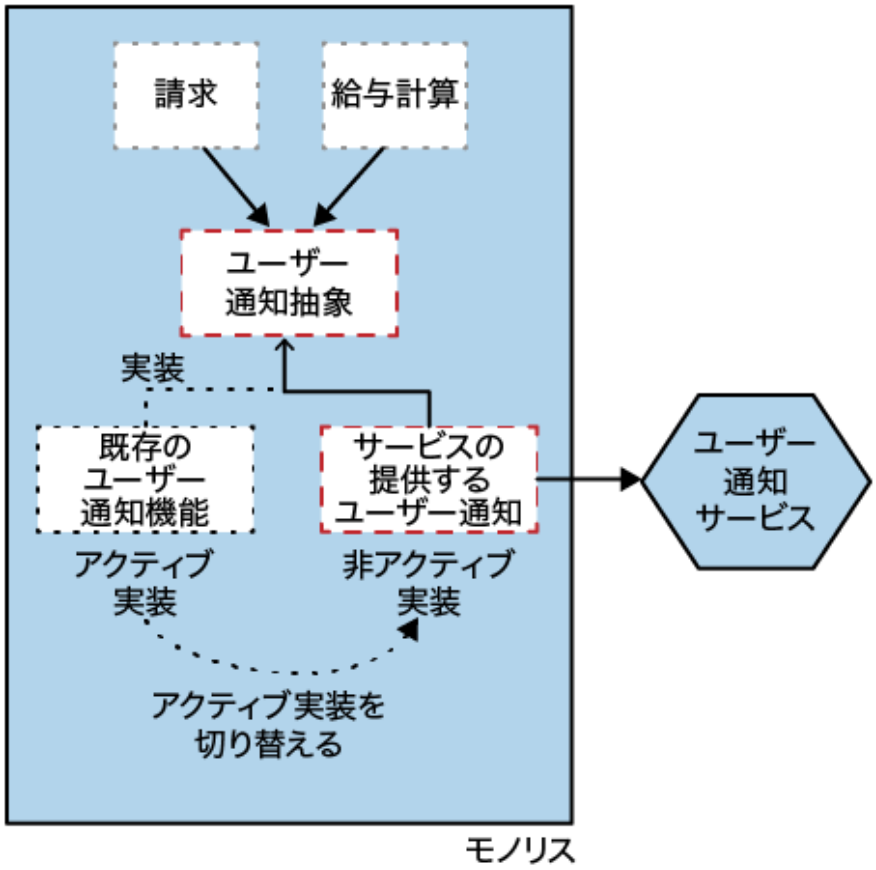

図3-25　ステップ4：アクティブな実装を切り替えて、新しいマイクロサービスを使用する

この作業は、「パターン：ストラングラーアプリケーション」のような、簡単に切り替えられる仕組みを用いて行うのが理想だ。そうすることで、問題が発生してもすぐに古い機能に戻せる。これに対する一般的な解決策は、**フィーチャーフラグ**を使うことだ。図3-26では、設定ファイルを使ってフラグを実装している。もし、フィーチャーフラグとその実装方法についてもっと知りたいのであれば、Pete Hodgsonが素晴らしい記事（http://bit.ly/2m316zB）[15]を書いている。

このステップでは、1つの抽象に対する2つの実装が機能的に同等であることを私たちは期待している。同等性の検証にはテストが使えるが、追加の検証を行うために、本番環境で**両方の**実装を使うという選択肢もある。このアイデアは「パターン：同時実行」で詳しく探る。

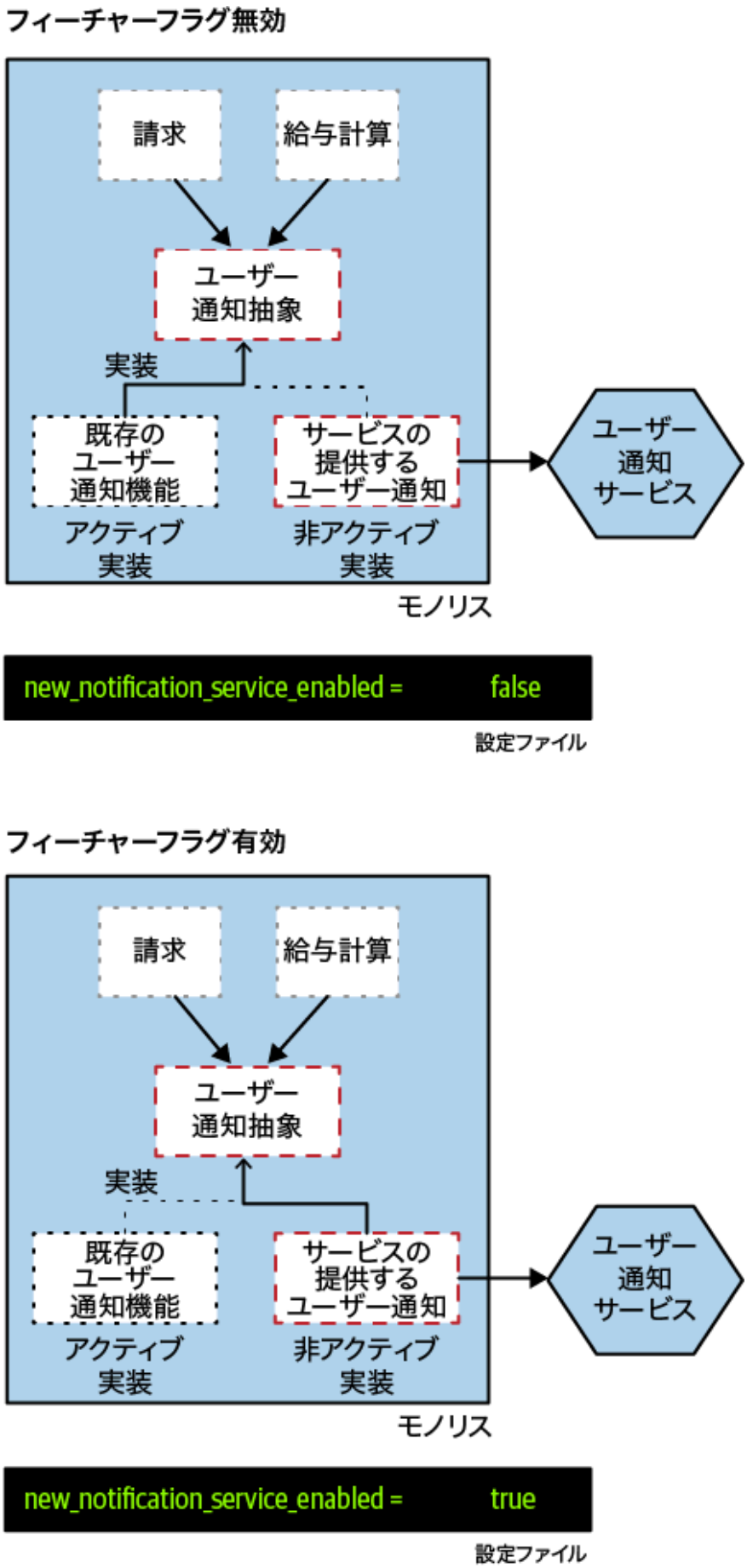

図3-26　ステップ4：フィーチャーフラグを使って実装を切り替える

ステップ5：後始末

新しいマイクロサービスがユーザー向けの全ての通知を提供するようになったら、後始末に注意を向けられるようになる。この時点では、古いユーザー通知機能はもう使われていない。ということは、次の明らかなステップはそれを削除することだ。図3-27にこれを示す。ついにモノリスが縮小し始めた！

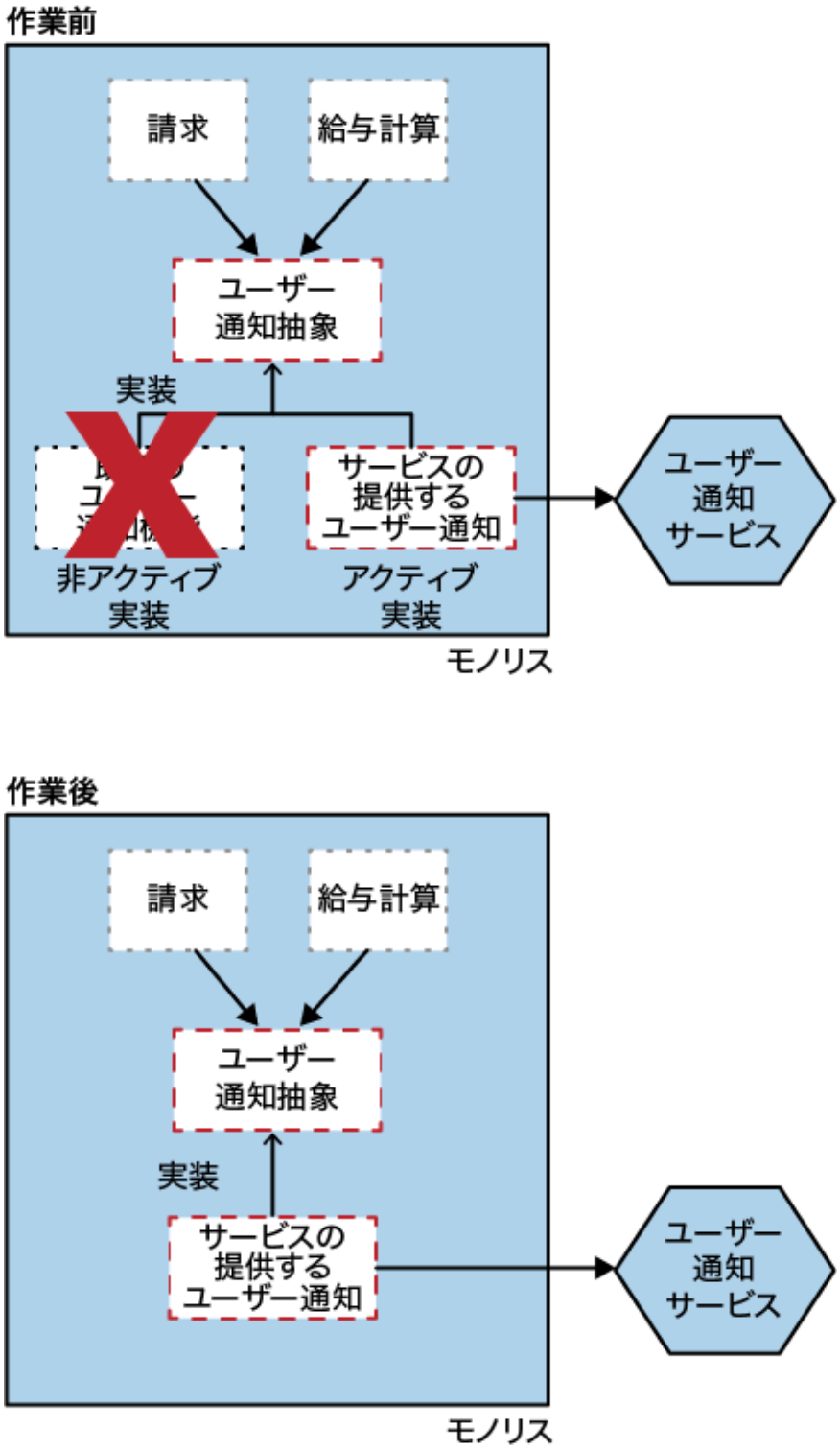

図3-27　ステップ5：古い実装を削除する

古い実装を削除する際には、フィーチャーフラグの切り替え処理を削除することにも意味がある。古いフラグが放置されてしまうというのは、フィーチャーフラグを使うと生じる問題の1つだ。そんなことはしてはいけない！　物事をシンプルに保つためにも、不要になったフラグは削除しよう。

古い実装を削除した後には、図3-28に示すように、抽象ポイント自体を削除するという選択肢がある。しかし、抽象を作成したことでコードベースを改善できたのだから、それをそのままにしておいた方が良いということもあるだろう。インターフェイスのような単純なものであれば、それを保持しても既存のコードベースへの影響はほとんどないはずだ。

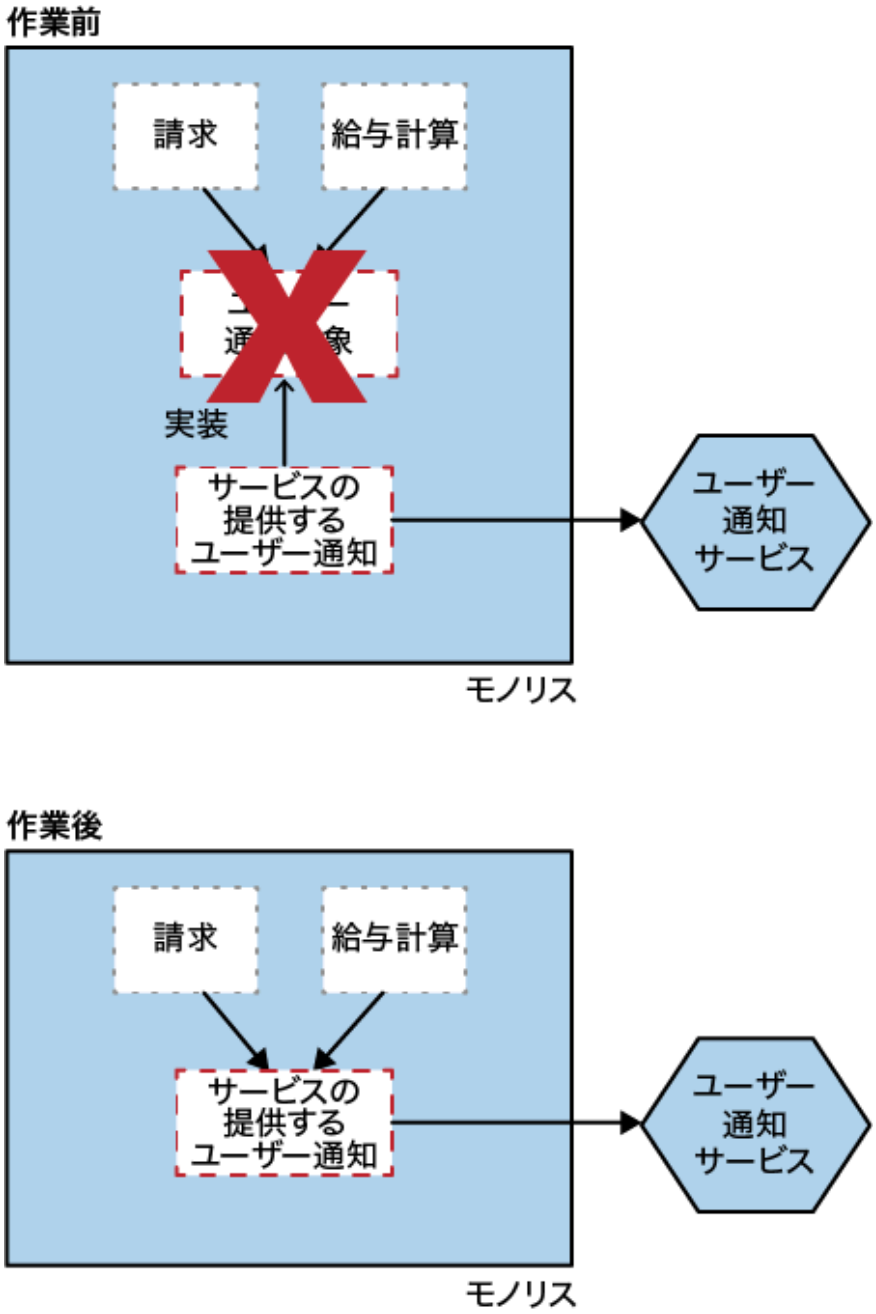

図3-28　ステップ6：（オプション）抽象ポイントを削除する

3.6.2　フォールバック機構として

新しいサービスが動作していない場合に以前の実装に戻すことは可能だ。しかし、それを自動的に行う方法はあるだろうか？　Steve Smithは、“Verify Branch By Abstraction（抽象化によるブランチの検証）”（http://bit.ly/2mLVevz）[31]という記事で、このパターンのバリエーションについて詳しく説明している。このバリエーションでは、実行時検証のステップも実装していく。例を図3-29に示す。このアイデアは、新しい実装への呼び出しが失敗した場合、古い実装を代わりに使えるというものだ。

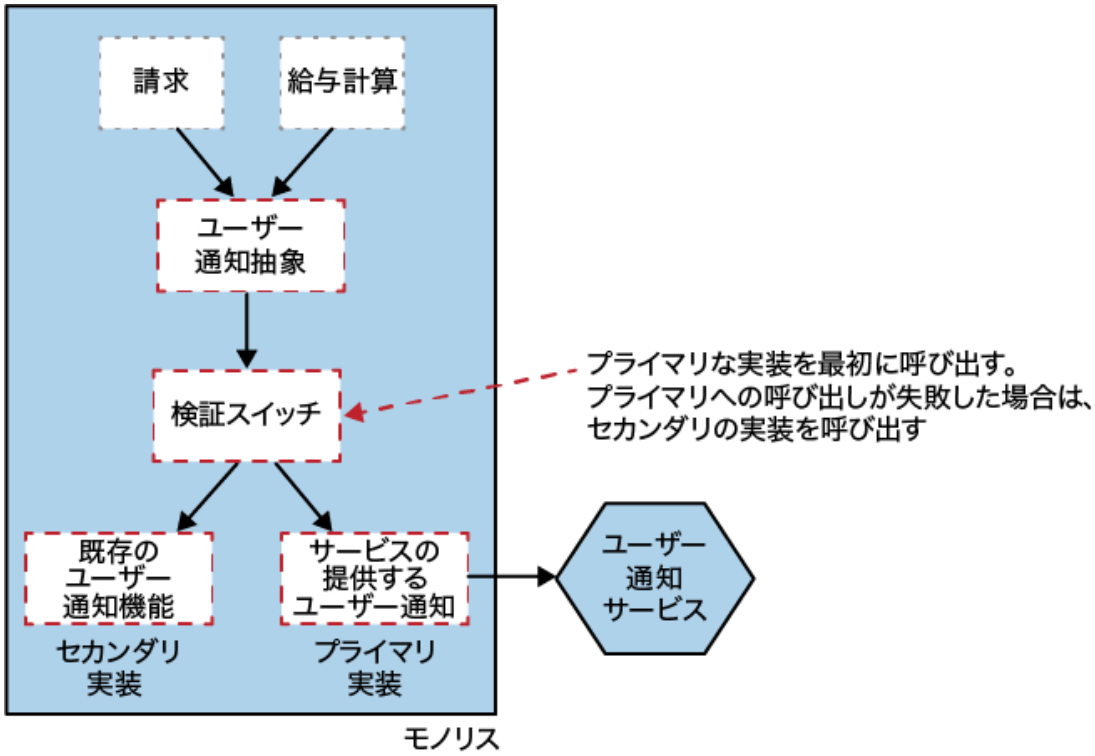

図3-29　抽象によるブランチの検証

このバリエーションは、コード面だけでなく、システム的にも明らかに複雑さを増す。事実上、両方の実装が有効になっていなければならず、それによってシステムの動作を理解するのがより困難になるからだ。2つの実装がステートフルなら、データの一貫性も考慮しなければならない。実装の切り替えを行う際には、データの一貫性は常に課題となるものだが、抽象によるブランチの検証パターンはリクエストごとに実装の切り替えを行えるために、両方の実装がアクセスできる一貫性のある共有データセットが必要になる。

このアイデアの詳細については、この後「パターン：同時実行」で詳しく探っていく。

3.6.3　使いどころ

抽象化によるブランチはとても汎用的なパターンだ。既存のコードベースを変更するのに時間がかかりそうな場合に便利だし、同僚を混乱させたくない場合にも有効だ。ほぼ全ての状況で、長命のコードブランチを使うよりもこちらの方が良い選択肢だろう。

マイクロサービスアーキテクチャへの移行では、ほとんどの場合、最初に「パターン：ストラングラーアプリケーション」を検討することにしている。多くの点で非常にシンプルだからだ。しかし、ここでの通知機能の例のように、それができない状況もある。

このパターンは、既存のシステムのコードを変更できることを前提としている。何らかの理由でそれができない場合には、他の選択肢を検討する必要があるかもしれない。そのうちのいくつかはこの章の残りで検討する。

3.7　パターン：同時実行

デプロイ前に新しい実装に対して行えるテストには限りがある。新しいマイクロサービスのリリース前検証は、通常のテストプロセスの中で、本番環境さながらに行われるだろう。一方で、本番環境で起こりうる全てのシナリオの網羅が困難であることは誰もが理解している。しかし、そこで利用可能なテクニックは他にもある。

「パターン：ストラングラーアプリケーション」と「パターン：抽象化によるブランチ」は、どちらも同じ機能の新旧の実装を本番環境で共存できるようにするものだ。これらのテクニックを使用すれば、モノリスにある古い実装と新しいマイクロサービスベースのソリューションのどちらかを実行できる。これらのテクニックは、新しいサービスベースの実装に切り替えるリスクを軽減するために、以前の実装にも素早く切り替えられるようになっている。

一方、**同時実行**では、古い実装と新しい実装のどちらかではなく、**両方**の実装を呼び出して結果を比較することで、それらが同等かを確認する。両方の実装を呼び出すものの、どちらか一方の実装が真の情報源としてみなされる。通常、新しい実装が信頼できると検証できるまでは、古い実装が真の情報源とみなされる。

このパターンは長年さまざまな形で使用されてきた。2つのシステムを並べて実行するためにこのパターンを使うことが一般的であるものの、同じ機能の2つの実装を比較するという観点から、1つのシステム内であってもこのパターンは同じように有用だ。

この手法は、新しい実装が既存の実装と同じ結果になるかどうかだけでなく、非機能的なパラメーターが許容範囲内で動作しているかを検証するためにも使用できる。たとえば、次のようなことを検証できる。新しいサービスは十分迅速に応答しているだろうか？　タイムアウトが多すぎないだろうか？

3.7.1　例：クレジットデリバティブの価格比較

何年も前、私は**クレジットデリバティブ**と呼ばれる金融商品の計算に使用されるプラットフォームを変更するプロジェクトに携わっていた。私が働いていた銀行は、提供しているさまざまなデリバティブが自分たちにとって賢明な取引かを確認する必要があった。この取引で儲かるのか？　取引はリスクが高すぎないか？　デリバティブが発行されれば、市場の状況も変化する。そのため、市場の状況が変化しても巨額の損失を被らないように、現在の取引の価値も評価する必要があった[†16]。

[†16] 私たちは、業界全体でこれがお粗末だったことを学んだ。2007年から2008年にかけての世界金融危機でクレジットデリバティブがどのような役割を果たしたかは『世紀の空売り』（文藝春秋）を読むといいだろう。私はしばしば、この業界で自分が果たした小さな役割を非常に後悔しながらふりかえる。私がそこで学んだのは、知らず知らずのうちにやってしまったことが、しばしばとても悲惨な結果を招くことがあるということだった。

私たちは、これらの重要な計算を行う既存のシステムをほぼ全面的に置き換えていた。大金に関わることと、一部の人々のボーナスが取引の価値に基づいていたという事実のために、この変更には大きな懸念があった。そこで、私たちは2つの計算セットを並べて実行し、結果を毎日比較することに決定した。価格付けイベントは、図3-30に示すように、両方のシステムが計算を実行するように複製が簡単なイベントを介してトリガーされた。

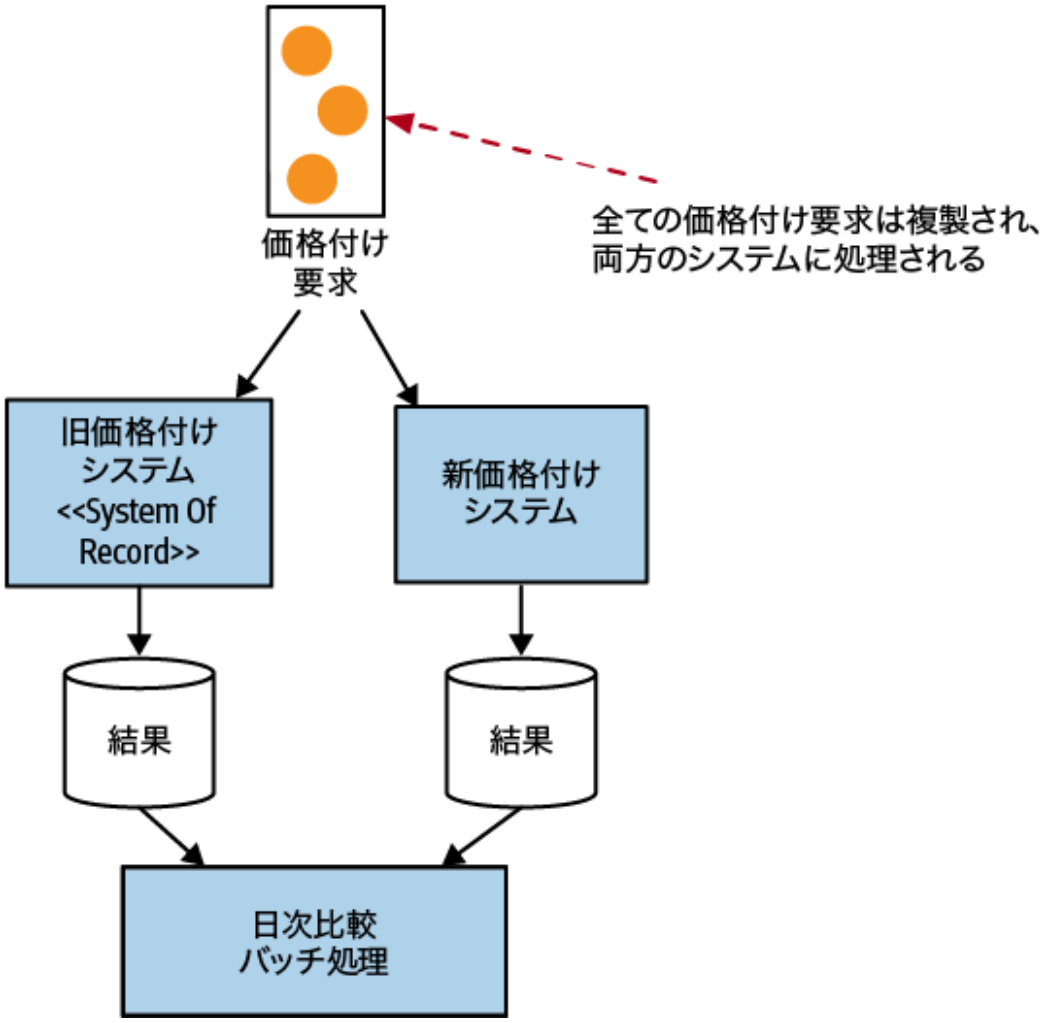

図3-30　同時実行の例。両方の価格付けシステムが呼び出され、結果がオフラインで比較される

私たちは毎朝結果のバッチ照合を行い、差異が発生した理由を説明する必要があった。この照合を行うプログラムも実際に作成した。結果をExcelのスプレッドシートにしたことで、差異について銀行にいる専門家たちと簡単に議論を行えた。

その結果、修正しなければならない問題がいくつか判明し、既存システムのバグが原因で発生した不一致も多く見つかった。結果が一致しなかったものの中には、新システムが吐き出した結果のほうが正しかったものもあったが、それが正しいことを説明するには、自分たちの仕事に誤りがないということを示す必要があった（結果をExcelの表にしたことで、この作業はかなり楽になった）。アナリストと一緒に座り、なぜ新システムの結果が正しいのかを基本原理から説明していったことを覚えている。

最終的に、1カ月後には新システムが真の情報源として計算に使われるようになり、しばらくして旧システムは引退した（旧システムで行われた計算の監査

を行う必要がある場合に備えて、さらに数カ月間は旧システムを使い続けた)。

3.7.2　例：Homegate社のリスティング

「3.3.7　例：FTP」で説明したように、Homegate社はリストのインポートを処理する新しいマイクロサービスを既存のモノリスと比較しながら、両方のリスティングインポートシステムを並行して実行した。このタイミングでは、顧客による1回のFTPアップロードで両方のシステムがトリガーされるようになっていた。やがて、新しいマイクロサービスが同等の動作を行えていることが確認されると、古いモノリスのFTPインポートは無効となった。

N-バージョンプログラミング

航空機のフライバイワイヤのような特定の安全上重要な制御システムの中にも、同時実行パターンの一種が存在すると言えるだろう。旅客機は、機械的な制御ではなく、ますますデジタル制御システムに依存するようになっている。航空機のフライバイワイヤでは、パイロットが制御装置を操作する際にケーブルを引っ張って舵を制御する代わりに、舵を回す量を決定する制御システムに入力が送られる。これらの制御システムは、送信されている信号を解釈し、適切なアクションを実行する必要がある。

これらの制御システムのバグは明らかに危険だ。欠陥の影響を相殺するために、状況によっては同じ機能の実装を複数並べて使用することもある。その場合には、同じサブシステムの実装全てに信号が送信され、それら全てが応答を送信する。これらの結果が比較され、通常は参加システムの中から定足数を満たすものを探して「正しい」ものが選択される。これは**N-バージョンプログラミング**[†17]として知られているテクニックだ。

このアプローチの最終目標は、この章で見てきた他のパターンとは異なり、実装を置き換えることではない。その代わりに、代替実装は互いに並んで存在し続け、うまくいけば、特定のサブシステムにおけるバグの影響を軽減することが期待される。

[†17] Liming Chen and Algirdas Avizienis, "N-Version Programming: A Fault-Tolerance Approach to Reliability of Software Operation," (the Twenty-Fifth International Symposium on Fault-Tolerant Computing, 1995).

3.7.3　検証テクニック

同時実行で、2つの実装の機能的同等性を比較したいとする。前述のクレジットデリバティブの価格付けを例に考えると、両方のバージョンを関数として扱える。つまり、同じ入力があれば、同じ出力が期待できるということだ。しかし、検証はそれだけでなく非機能的な側面についても行えるし、そうすべきだ。ネットワークの境界を越えて行われる呼び出しは大きなレイテンシーをもたらし、タイムアウトやパーティションなどによるリクエストの損失の原因となることがある。したがって、新しいマイクロサービスへの呼び出しが適時に行われること、そして許容できる失敗率であることの保証も、私たちは検証プロセスに含める必要がある。

3.7.4　スパイを使う

先ほどの通知の場合なら、顧客に2度もメールを送りたくはないだろう。そうした状況では、**スパイ**が使える。スパイとは、機能の代理を務めることで、ある特定のことが行われたかを後から検証できるようにするユニットテストのパターンだ[†18]。スパイは、機能のある部分に置き換わって代理を務め、それをスタブ化する。

[†18] 訳注：テストダブル（テスト対象が依存するコンポーネントを、テストの観点で等価であるコンポーネントに置き換えるパターン）の一種。テストスパイとも呼ばれる。http://xunitpatterns.com/Test%20Spy.html

通知機能の例であれば、メール送信に実際に使用される低レベルのコードを置き換えるのにスパイを使用できる。これを図3-31に示す。同時実行のフェーズでこのスパイを使うことで、新しい通知サービスの通知送信が呼び出されたときに、この副作用（電子メールの送信）がトリガーされるかを検証できる。

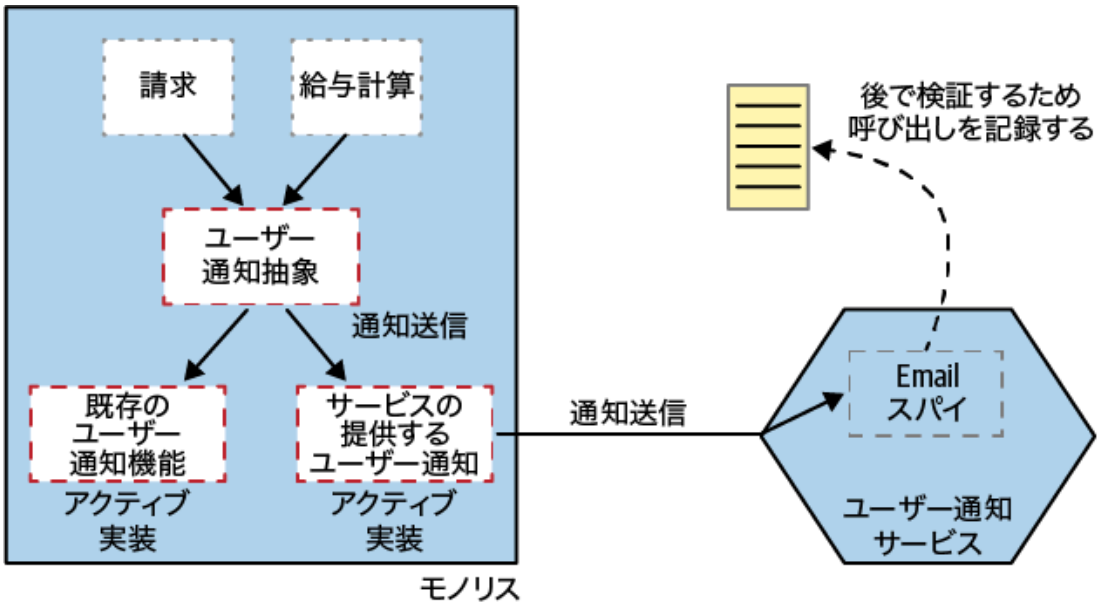

図3-31　スパイを使用して、同時実行中にメールが送信されたことを確認する

モノリス内部でスパイを使用して、リモート呼び出しを行うコードがサービス呼び出しを行わないようにすることもできるだろう。しかし、それはおそらく望むものではない。タイムアウト、障害、新しい通知サービスへの全般的なレイテンシーが実際に問題を引き起こすかどうかを理解するためにリモート呼び出しの影響を考慮したいからだ。

スパイが別のプロセスで実行されているという事実は、検証プロセスのタイミングを複雑にする。最初のリクエストの範囲内でこれを実行時検証するには、最初の呼び出しが通知サービスに送信された後に検証を実行できるよう、通知サービスでメソッドを公開する必要があるだろう。これはかなりの作業量になる可能性があり、そして多くの場合は実行時検証を必要としない。プロセス外のスパイによる検証のモデルでもっとよくあるのは、日常的に検証を行えるように相互作用を記録することだろう。通知サービスへの呼び出しの代わりにスパイを使用すれば、検証は明らかにずっと簡単になるし、テストの回数も減る。ただし、スパイに置き換える機能が増えれば増えるほど、実際にテストされる機能は少なくなる。

3.7.5　GitHub Scientist

GitHubのScientistライブラリ[†19]は、このパターンをコードレベルで実装するのに役立つ注目すべきライブラリだ。Scientistは、ある機能に対する複数の実装を並べて動作できるようにし、新しい実装が適切に動いているかを確かめるのに役立つ情報を記録するためのRubyライブラリだ。私自身は使ったことはないが、このようなライブラリがあればマイクロサービスベースの新しい機

能を既存のシステムと比較して検証するのにとても役立つだろう。Scientistには、Java、.NET、Python、Node.jsをはじめとする、複数の言語向けの移植がある。

[†19] https://github.com/github/scientist

3.7.6　ダークローンチとカナリアリリース

同時実行は、**カナリアリリース**とは異なることに注意してほしい。カナリアリリースでは、ユーザーの一部を新しい機能に誘導し、残りのユーザーは引き続き古い実装を体験する。この考えは、新しいシステムに問題があっても、影響を一部のリクエストのみに抑えられるというものだ。それと異なり、同時実行は**両方の**実装を呼び出す。

もう一つの関連するテクニックとして、**ダークローンチ**と呼ばれるものがある。ダークローンチでは、新しい機能をデプロイしてテストするが、新しい機能はユーザーには見えないようにする。したがって、同時実行はダークローンチを実装する1つの方法といえる。実行中のシステムを切り替えるまで、「新しい」機能はユーザーには見えないからだ。

ダークローンチ、同時実行、カナリアリリースといったテクニックは、新しい機能が正しく機能していることを確認し、そうでなかった場合の影響を軽減するためのものだ。これらのテクニックは全て、**プログレッシブデリバリー**と呼ばれるカテゴリに含まれる。これは、James Governorによる造語[†20]で、ソフトウェアの微妙に異なるバージョンを公開し、その有効性を検証すると共に問題が発生した場合の影響を軽減しながら、より迅速にソフトウェアをリリースすることを支援する手法の総称だ。

[†20] http://bit.ly/2lZjrxK

3.7.7　使いどころ

同時実行の実装は決して簡単ではない。一般的には、機能を変更するリスクが高い場合にのみ行われる。4章では、このパターンが診療記録に使われている

例を見ていく。このパターンを使うかどうかは、私自身はかなり慎重に検討する。私がこのパターンを使用したことがあるのは1、2回だけだが、そのような状況では非常に役立った。

3.8　パターン：デコレーティングコラボレーター

モノリス内部の挙動に基づいて動作をトリガーしたいが、モノリス自体を変更できない場合はどうしたらよいだろうか？　そうしたときには、**デコレーティングコラボレーター**がとても役立つ。よく知られているように、デコレーターパターン[†21]を使うと、基礎となるものが何かを知ることなしに新しい機能をそこに追加できる。デコレーターを使うことで、基礎となるモノリスを変更しなくても、モノリスが直接サービスを呼び出しているように見せかけられる。

[†21] 訳注：https://ja.wikipedia.org/wiki/Decorator_パターン

このパターンでは、モノリスに到達する前に呼び出しを傍受するのではなく、呼び出しは通常どおり行えるようにする。そして、その呼び出しの結果に基づいて、何であれ選択したコラボレーションメカニズムを通して外部のマイクロサービスを呼び出せるようにする。Music Corpの例を使って、このアイデアを詳しく探っていこう。

3.8.1　例：ポイントプログラム

Music Corpは、お客さまを第一とする企業だ。そこで、注文に基づいて顧客がポイントを獲得できる機能を追加しようと考えた。しかし、現在の注文処理はとても複雑なため、今すぐには変更したくない。そこで、注文機能は既存のモノリスのままとしつつ、プロキシを使用してこれらの呼び出しを傍受し、その結果からどのくらいポイントを付与するかを決定することにする。これを図3-32に示す。

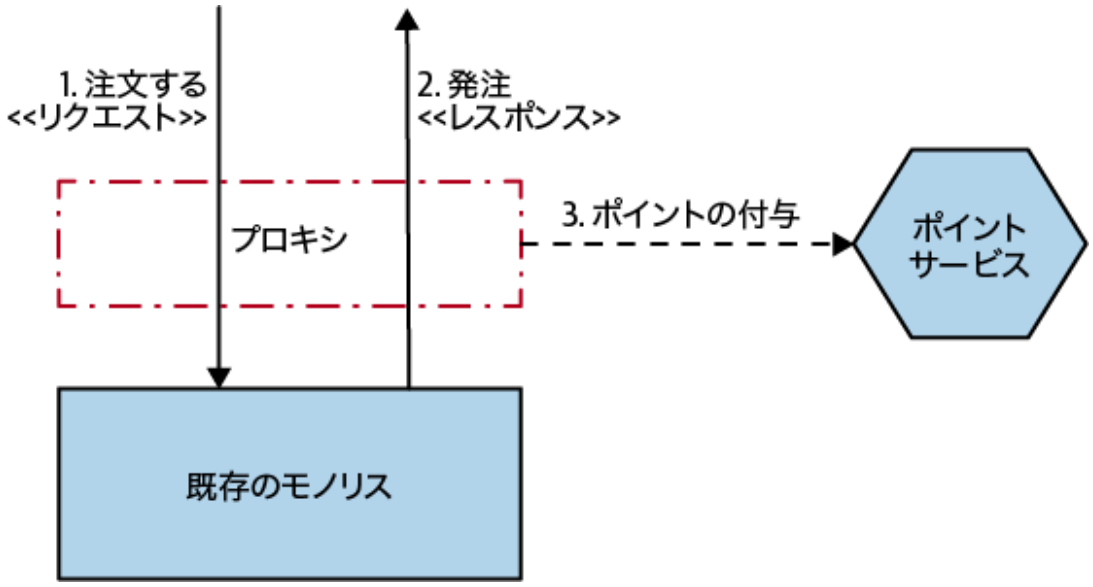

図3-32　注文が正常に行われると、プロキシがポイントサービスを呼び出して顧客にポイントを付与する

ストラングラーパターンにおけるプロキシはかなり単純だった。このパターンでのプロキシは、それに比べてさらに多くの「スマートさ」を実現しなくてはならない。プロキシは、それ自身で新しいマイクロサービスを呼び出し、その結果を顧客にトンネルする必要がある。そのため、プロキシ自身が担うことになる複雑さについては、引き続き用心しよう。プロキシにコードを追加すればするほど、それ自体がマイクロサービスになってしまい、これまでに説明した技術的な課題全てがそこに発生することになる。

もう一つの潜在的な課題は、マイクロサービスの呼び出しを行うには、受信リクエストから十分な情報を得る必要があるということだ。たとえば、注文価格に基づいてポイントを付与したいが、注文価格が注文を行うリクエストやレスポンスからは明らかではない場合には、追加の情報を調べなければならない可能性がある。そのためには、図3-33に示すように、必要な情報を抽出するためにモノリスを呼び出す必要があるかもしれない。

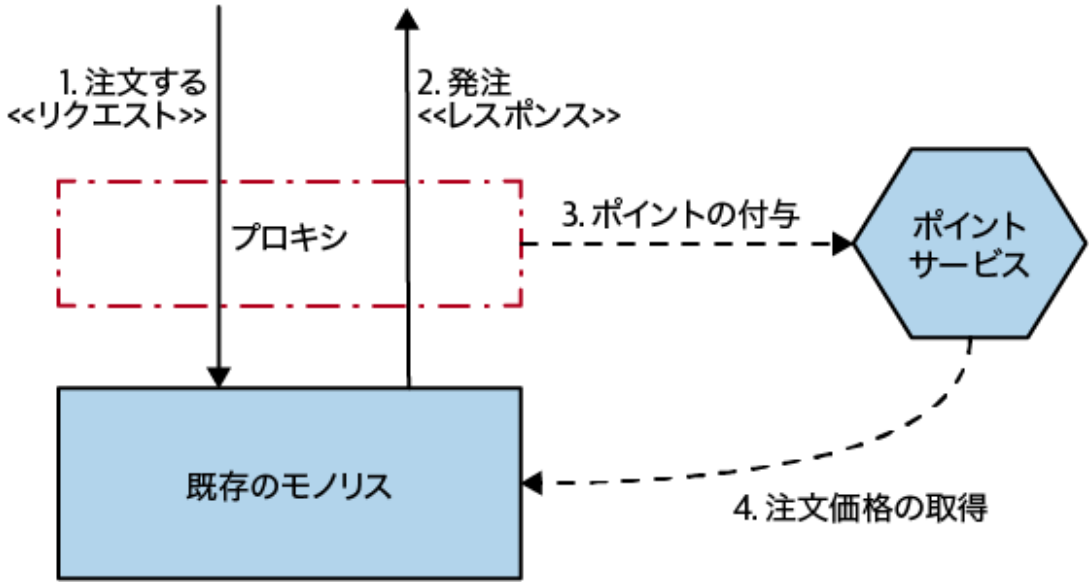

図3-33　ポイントサービスでは、追加の注文詳細を読み込んで、獲得ポイントを算出する必要がある

この呼び出しがさらなる負荷を生じさせるかもしれず、明らかに循環的な依存関係を導入することを考えると、注文が完了したときに必要な情報を提供するようにモノリスを変更する方が良いかもしれない。しかし、その場合にはモノリスのコードを変更する必要があるか、「パターン：変更データキャプチャ」のようなより侵略的なテクニックを使用する必要がある。

3.8.2　使いどころ

シンプルに保っている限りは、このパターンは「パターン：変更データキャプチャ」よりもエレガントで結合の少ないアプローチだ。このパターンは、受信リクエストやモノリスからのレスポンスから必要な情報を抽出できる場合には最適だ。もし、新しいサービスに対して適切な呼び出しを行うためにより多くの情報が必要な場合、この実装はより複雑で絡み合ったものになる。モノリスへの要求とモノリスからの応答に必要な情報が含まれていない場合には、このパターンを使用する前によく考える必要があるだろう。

3.9　パターン：変更データキャプチャ

変更データキャプチャは、モノリスに行われた呼び出しを傍受して動作させようとするのではなく、データストアで行われた変更に反応するパターンだ。変更データキャプチャが動作するには、基礎となるキャプチャシステムをモノリスのデータストアに結合する必要がある。これは、このパターンでは避けようのない課題だ。

3.9.1　例：ポイントカードの発行

ユーザーがサインアップしたときにポイントカードを印刷する機能を統合したいとする。現時点で、ポイント会員が作成されるのは、顧客が登録されたタイミングだ。図3-34に示すように、登録処理がモノリスから返ってきたときに分かるのは、顧客が正常に登録されたことだけだ。ポイントカードを印刷するには、顧客に関するより詳細な情報が必要となる。それには、「パターン：デコレーティングコラボレーター」を使うなどして、その動作を上流に差し込む必要があるが、それはずっと難しい。呼び出しが返ってきたら必要な情報を取得する追加のクエリをモノリスに発行する必要があるものの、その情報がAPIを介して公開されているかどうかは怪しいからだ。

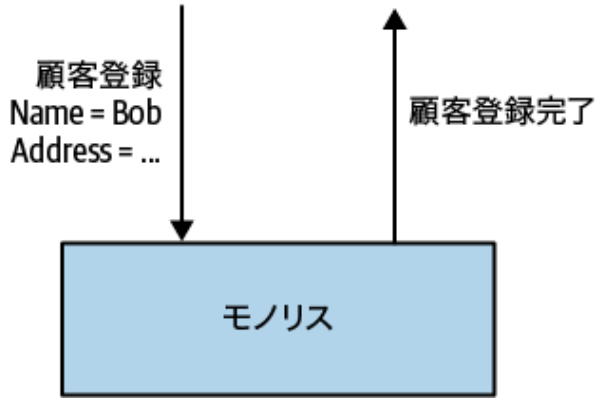

図3-34　顧客が登録されたときに、モノリスは多くの情報を共有しない

ここでは、代わりに変更データキャプチャを使用することにする。ポイント会員テーブルに対する挿入を検出し、挿入時に新しいポイントカード印刷サービスを呼び出すようにする。図3-35にこれを示す。この例では、ポイント会員作成イベントを発行することにした。印刷プロセスはバッチでうまく動作するので、これでメッセージブローカーによって処理される印刷物のリストを構築できる。

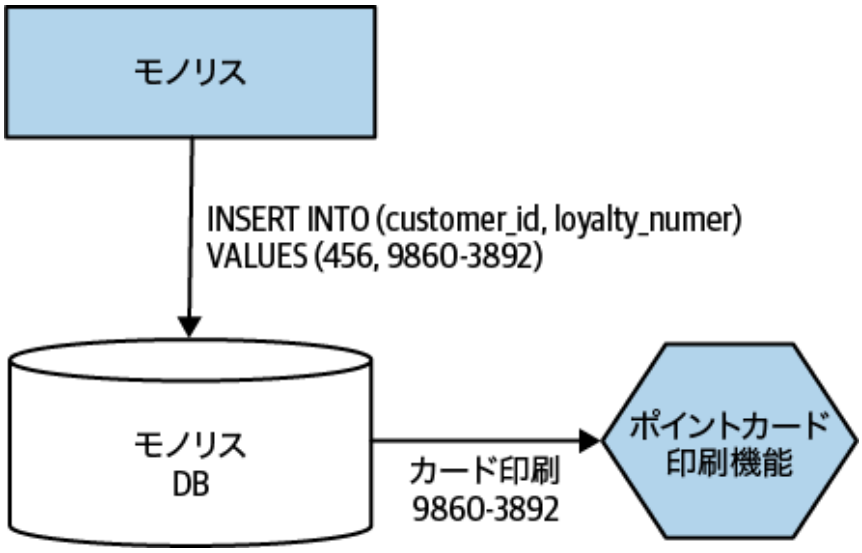

図3-35　変更データキャプチャを使用して新しい印刷サービスを呼び出す

3.9.2　変更データキャプチャを実装する

変更データの取り込みにはさまざまな技術を使用できるが、その全てが複雑さ、信頼性、適時性の点で異なるトレードオフを持っている。いくつかの選択肢を見ていこう。

データベーストリガー

ほとんどのリレーショナルデータベースでは、データが変更されたときにカスタム動作をトリガーできる。トリガーがどのように定義され、何をトリガーとできるかは、厳密にはそれぞれのデータベースで異なるものの、最新のリレーショナルデータベースは全て、何らかの方法でこれらをサポートしている。図3-36の例では、ポイント会員テーブルに`INSERT`されるたびにサービスが呼び出される。

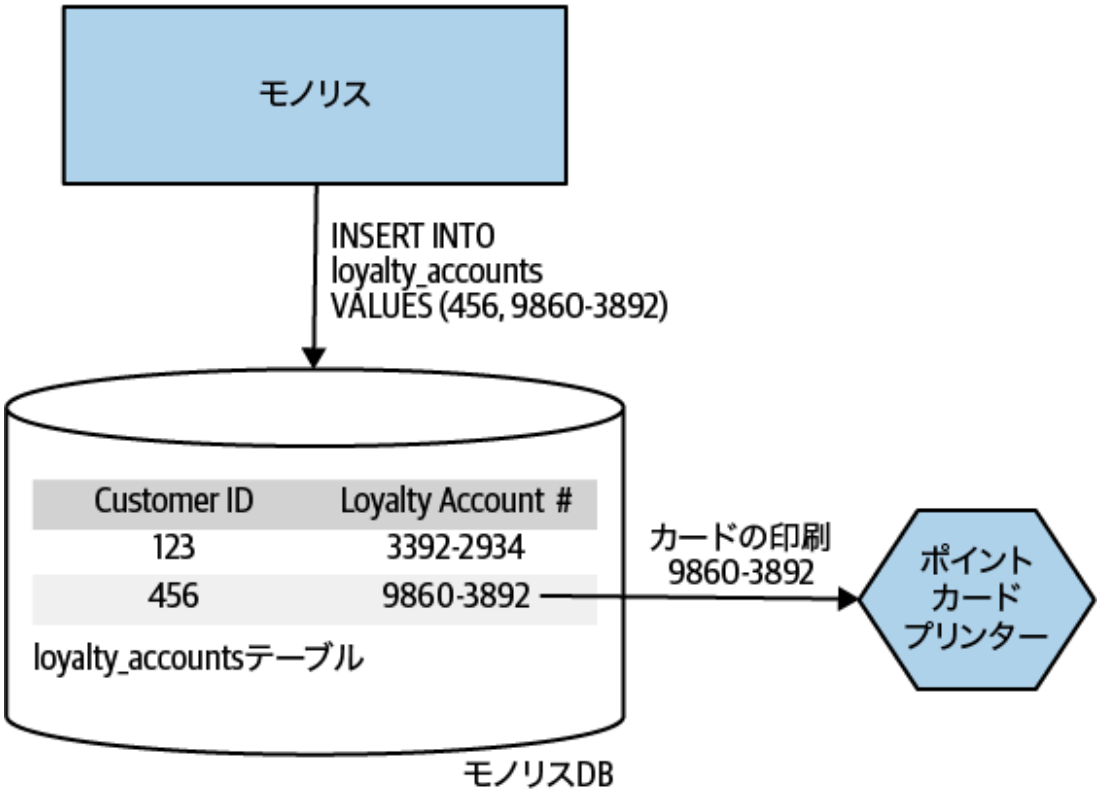

図3-36　データベーストリガーを使用して、データが挿入されたときにマイクロサービスを呼び出す

トリガーは、他のストアドプロシージャと同様にデータベース自体にインストールする必要がある。また、これらのトリガーでできることには制限があるかもしれない。少なくともOracleではWebサービスやカスタムのJavaコードを呼び出せる。

一見すると、これは非常に簡単なことのように思える。他のソフトウェアを動かす必要も、新しい技術を導入する必要もない。しかし、ストアドプロシージャのように、データベーストリガーは一歩間違えると厄介な事態を引き起こす。

私の友人であるRandy Shoupは、かつて「1つか2つのデータベーストリガーを持つことは悪くない。しかし、それを利用してシステム全体を構築するのは、あり得ない考えだ」と語っていた。しかし、これは実際によく起こる問題だ。トリガーの数が多ければ多いほど、システムが実際にどのように動作するかを理解するのが難しくなる。多くの場合、問題はデータベーストリガーのツールと変更管理にある。あまりにもデータベーストリガーを多く使いすぎると、アプリケーションがバロック様式の建築物になってしまうことがある。

したがって、データベーストリガーを使用する場合でも、使用は**控えめに**しよう。

トランザクションログ監視

ほとんどのデータベース、それも主流のトランザクショナルデータベースの内部には、間違いなくトランザクションログが存在する。これは通常、ファイルであり、そこには行われた変更の全記録が書き込まれている。変更データキャプチャ用の最も洗練されたツールは、このトランザクションログを利用する傾向が強い。

これらのシステムは独立したプロセスとして実行され、図3-37で示すように、既存のデータベースとの相互作用は、このトランザクションログを介してのみ行われる。ここで注目すべきは、コミットされたトランザクションのみがトランザクションログに表示されることだ（これが要点のようなものだ）。

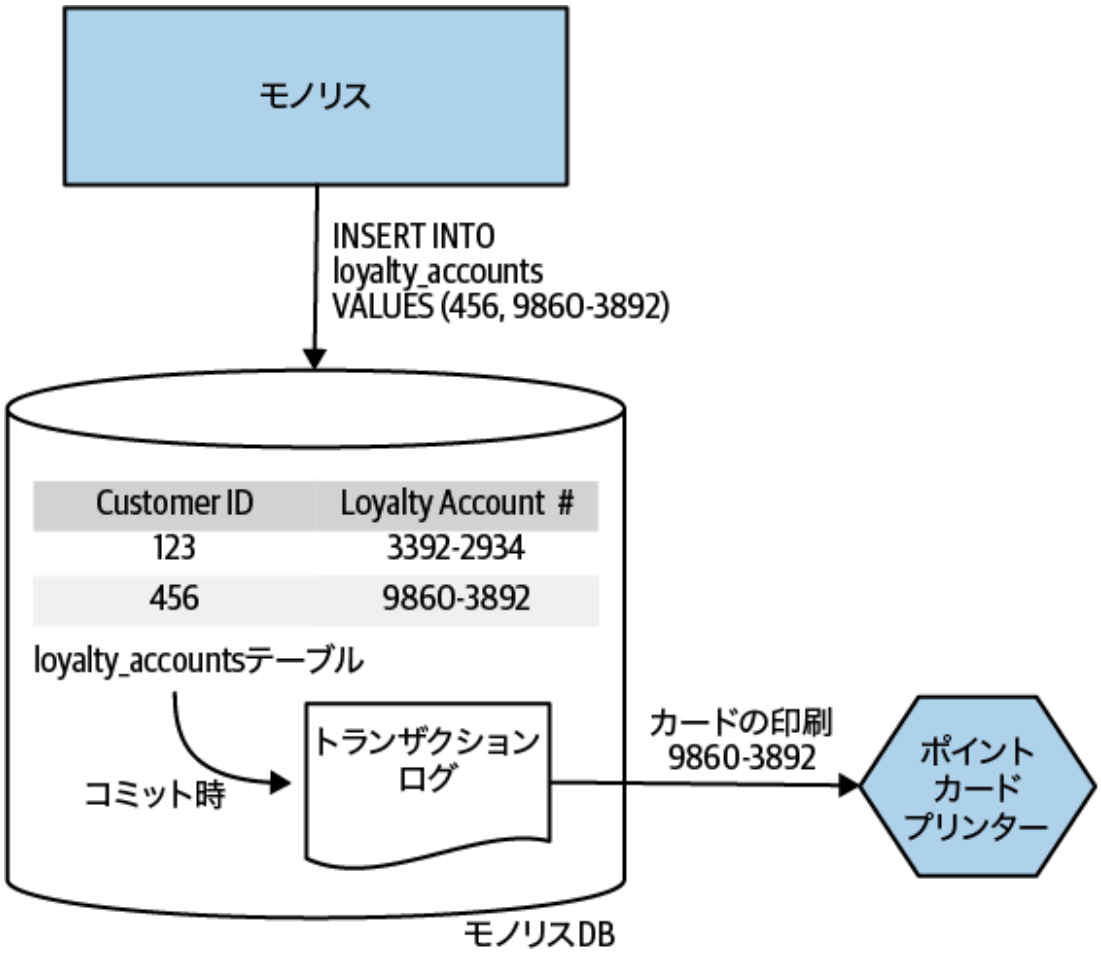

図3-37　基礎となるトランザクションログを利用する変更データキャプチャシステム

これらのツールは、基礎となるトランザクションログの形式を理解する必要があり、そうした形式は通常、データベースの種類によって異なる。そのため、変更データキャプチャに利用できるツールがどのようなものかは、使用するデータベースによって異なる。この分野には膨大な数のツールが存在するが、その多くはデータレプリケーションをサポートするために使用される。また、トランザクションログへの変更をメッセージにマッピングし、メッセージブローカーに配置するように設計されたソリューションも数多くある。これは、マイクロサービスが非同期である場合に非常に便利だ。

制限はさておき、多くの点で、これは変更データキャプチャを実装するための最もすっきりしたソリューションだ。トランザクションログ自体は、基礎となるデータへの変更のみを表示するので、何が変更されたかを調べることを心配する必要はない。ツールはデータベース自体の外で動作し、トランザクションログのレプリカから実行できるので、結合や競合に関する懸念が少なくなる。

バッチによる差分コピー

おそらく最も単純なアプローチは、定期的なスケジュールで問題のデータベースをスキャンしてどのようなデータが変更されたかを調べ、そのデータをコピーするプログラムを書くことだ。これらのジョブは、`cron`などのバッチスケジューリングツールを使って実行されることが多い。

主な問題は、バッチによるコピーが最後に実行されてから実際にどのようなデータが変更されたかを調べる部分にある。スキーマ設計によってこれが明らかになる場合もあれば、そうでない場合もある。いくつかのデータベースでは、データベースの一部がいつ変更されたかをテーブルのメタデータとして確認できるようになっているが、これは万能とは言いがたい。中には、行レベルで情報を得たいのに、テーブルレベルでしか変更タイムスタンプが得られないものもある。自分でこれらのタイムスタンプを追加することもできるが、これはかなりの作業を追加することになる。

3.9.3　使いどころ

変更データキャプチャは、特にデータを複製する必要がある場合に便利な汎用パターンだ（詳しくは4章で説明する）。マイクロサービスへの移行の場合、モノリス内のデータの変化に反応する必要があるものの、システムの境界でストラングラーやデコレーターを使ってそれを傍受できず、基礎となるコードベースを変更することもできないような場面が、このパターンの使いどころだ。

一般的に、このパターンのいくつかの実装には課題があるため、私はこのパターンの使用を最小限に抑えるようにしている。データベーストリガーには欠点があり、トランザクションログから動作する本格的な変更データキャプチャツールは、ソリューションに大きな複雑さを加える可能性があるからだ。それでも、これらの潜在的な課題を理解していれば、変更データキャプチャは自由に使える便利なツールになる。

3.10　この章のまとめ

ここまで見てきたように、既存のコードベースを段階的に分解し、マイクロサービスの世界へとゆっくりと足を踏み入れることを可能にする、膨大な数のテクニックが存在する。私の経験では、ほとんどの場合でさまざまなアプローチを組み合わせて使うことになる。1つのテクニックだけで全ての状況を処理できることはまれだ。願わくはさまざまなアプローチがあるということ、そして、どのテクニックが最適なのかを見極めるのに十分な情報をあなたに提供できていればと思う。

ここまで、マイクロサービスアーキテクチャに移行する際の難問の1つをごまかしてきていた。それはデータだ。しかし、もう先延ばしにはできない。4章では、データを移行する方法、そして、データベースを分解する方法について探っていこう。